

A PROJECT REPORT
On
NETWORK SECURITY SYSTEM: A FRAMEWORK TO DETECT
ATTACK

Submitted by

Rishav Kumar (10800222043)
Vivek Kumar (10800222038)
Md. Samir Ansari (10800223188)
Ehtesham Khan (10800223181)

Submitted to Asansol Engineering College in partial fulfillment of the
Requirements for the degree of
Bachelor of Technology
(Information Technology)
Under the Guidance
of
Prof. Ganga Bhattacharjee
(Assistant Professor)



Information Technology
Asansol Engineering College
Asansol

Affiliated to
MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY

07/2026



ASANSOL ENGINEERING COLLEGE

Kanyapur, Vivekananda Sarani, Asansol, Paschim Bardhaman, West Bengal - 713305
Phone: 225-3057, 225-2108 Telefax: (0341) 225-6334
E-mail: principal.aecwb@gmail.com Website: www.aecwb.edu.in

CERTIFICATE

Certified that this project report on "**Network Security System: A Framework to detect Phishing Attack**" is the bonafide work of "**Rishav Kumar (10800222043), Vivek Kumar (10800222038), Md. Samir Ansari (10800223188), Ehtesham Khan (10800223181)**" who carried out the project work under my supervision.

Prof. Ganga Bhattacharjee

Assistant Professor

Dr. Vedatrayee Chatterjee

Head of the Department
Dept. of Information Technology

Information Technology
Asansol Engineering College
Asansol

ACKNOWLEDGEMENT

It is our great privilege to express our profound and sincere gratitude to our **Project Supervisor Prof. Ganga Bhattacharjee, Assistant Professor** for providing us with cooperative and invaluable guidance at every stage of this project. Their advice on structuring a machine-learning system responsibly — insisting that every design decision be verified against real URLs rather than accepted on faith — shaped the engineering discipline behind this work far more than any single feature or model choice.

We would also like to pay our heartliest thanks and gratitude to Dr. Vedatrayee Chatterjee, Hod and all the faculty members of the Dept. of Information Technology, Asansol Engineering College for various suggestions being provided in attaining success in our work.

Finally, we would like to express our deep sense of gratitude to our parents for their constant motivation and support throughout our work.

.....

(Rishav Kumar)

.....

(Vivek Kumar)

.....

(Md. Samir Ansari)

.....

(Ehtesham Khan)

Date: 10/07/2026

Place: Asansol

4th Year

Information Technology

Abstract

Phishing is one of the most common cyber threats, where attackers create fraudulent websites to steal sensitive information such as usernames, passwords, and financial details. Traditional phishing detection methods often rely on blacklists, webpage analysis, or third-party reputation services, which may fail to detect newly created phishing websites and require internet connectivity. This project, **Network Security System: A Framework to Detect Attack**, presents a machine learning-based framework for real-time phishing URL detection using only the textual characteristics of URLs.

The proposed system combines rule-based techniques, including allowlists, blocklists, and typosquatting detection, with an **XGBoost** machine learning model to accurately classify URLs as safe or phishing. The model is trained using the **PhiUSIIL Phishing URL Dataset**, which contains over **235,000 labeled URLs**. Various lexical and structural features are extracted from URLs along with character-level TF-IDF representations to improve classification performance. The framework is implemented using **Python**, **FastAPI**, and modern web technologies, and also includes a **Manifest V3 browser extension** that provides real-time protection while browsing.

Experimental results demonstrate that the proposed framework achieves high detection accuracy with low response time while reducing false positives through effective feature engineering and data augmentation. This project offers a fast, scalable, and reliable solution for phishing URL detection, contributing to improved cybersecurity and safer web browsing.

Contents

CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
Abstract	iv
LIST OF TABLES	ix
LIST OF FIGURES	x
1. Preface	1
1.1 Introduction	1
1.2 Motivation of the Project	2
1.2.1 The scale and persistence of the phishing problem	2
1.2.2 The limitations of purely list-based and purely content-based defenses	2
1.2.3 An academic interest in honest, verifiable machine-learning evaluation	3
1.3 Problem Statement	3
1.4 Objectives of the Project	4
2. Literature Review	5
2.1 Review of Existing Research	5
2.1.1 Rule-based and heuristic detection	5
2.1.2 Classical machine learning over hand-engineered URL/lexical features	5
2.1.3 Deep representation learning over the raw URL string	6
2.1.4 The dataset landscape, and why PhiUSIIL	6
2.2 Existing Systems and Limitations	7
2.2.1 Community and vendor blocklists (PhishTank, OpenPhish, URLhaus)	7
2.2.2 Browser- and OS-integrated reputation systems (Google Safe Browsing, Microsoft SmartScreen)	7
2.2.3 Content-rendering and visual-similarity detectors	7
2.2.4 Browser extensions built on third-party reputation APIs	8
2.3 Comparative Analysis Table	8

3. Dataset Analysis And Requirements Study	10
3.1 Functional Requirements	10
3.2 Non-Functional Requirements	11
3.3 Dataset Overview	11
3.4 Attribute Description	12
3.4.1 Attributes supplied by the raw dataset	12
3.4.2 Attributes engineered by this project (core/features.py)	13
4. Related Theories and Algorithms	15
4.1 Fundamental Theories Underlying the Work	15
4.1.1 Binary classification and the bias–variance trade-off	15
4.1.2 Ensemble learning: bagging versus boosting	15
4.1.3 Information theory: Shannon entropy as a randomness signal	16
4.1.4 Text representation theory: bag-of-character-n-grams and TF-IDF	16
4.1.5 Edit-distance theory and Unicode confusables	16
4.1.6 Decision thresholds and the precision–recall trade-off	17
4.2 Fundamental Algorithms	17
4.2.1 Decision trees (CART)	17
4.2.2 Random Forest	17
4.2.3 Gradient boosting and XGBoost	18
4.2.4 Logistic Regression (baseline comparison model)	18
4.2.5 Damerau–Levenshtein edit distance	19
4.2.6 Unicode NFKC normalisation	19
4.3 Summary	20
5. Proposed Framework and Methodology	21
5.1.1 Proposed Framework Architecture	21
5.1.2 Data Flow Architecture	22
5.1.3 Data Flow Diagram (Level 0)	22

5.1.4 Data Flow Diagram (Level 1)	23
5.2.1 System Workflow Pipeline	24
5.2.2 User Interface Snapshots	24
5.3 Data Preparation Strategy	27
5.3.1 The www.-prefix artefact	27
5.3.2 The bare-homepage / trailing-slash artefact	28
5.3.3 Augmentation to counteract the "any real path = phishing" shortcut	28
5.4 Feature Optimization Methodology	29
5.5 Machine Learning Model Selection	29
5.6 Performance Evaluation Criteria	30
6. TECHNOLOGY USED	31
6.1 Programming Language	31
6.2 Backend Framework — FastAPI	31
6.3 Data Validation — Pydantic	32
6.4 ASGI Server — Uvicorn	32
6.5 Machine Learning and Data Processing Stack	32
6.6 Frontend Technologies	32
6.7 Browser Extension — Manifest V3	33
6.8 Testing Stack	33
6.9 Deployment and Version Control	33
6.10 Dataset Source	33
6.11 Summary of Technology Choices	34
7. FRAMEWORK IMPLEMENTATION AND EXPERIMENTAL RESULTS	35
7.1 Development Environment	35
7.1.1 Iterative audit-and-upgrade development methodology	35
7.2 Dataset Loading and Inspection	36
7.3 Data Preparation Results	37

7.4 Feature Selection Results	37
7.5 Machine Learning Model Development and Evaluation	38
7.6 Performance Assessment	39
7.6.1 Before/after: closing the held-out generalisation gap	39
7.7 Model Comparison and Discussion	40
7.8 Best Model Identification and Result Interpretation.....	41
7.8.1 Automated regression test suite	41
8. CONCLUSION AND FUTURE WORK	43
8.1 Project Conclusions	43
8.2 Key Outcomes.....	43
8.3 Current Constraints	44
8.4 Future Scopes.....	45
9. REFERENCES	46

LIST OF TABLES

2.1	Comparative analysis of phishing-detection approaches	9
3.1	Functional requirements	10
3.2	Non-functional requirements	11
3.3	PhiUSIIL dataset composition	12
3.4	Attribute groups supplied by the PhiUSIIL dataset	13
3.5	The 34 engineered numeric features (+1 text field) used by the model	13
4.1	Theory-to-implementation mapping	20
5.1	Evaluation metrics used throughout Chapter 7	30
6.1	Machine learning / data processing libraries	32
6.2	Full technology stack by layer	34
7.1	Development and evaluation environment	35
7.2	Confirmed defects found and fixed during the audit pass (Phase 1)	36
7.3	Training-data pipeline row counts	37
7.4	Top 15 features by XGBoost gain-based importance	38
7.5	Head-to-head model comparison	39
7.6	Impact of augmentation-set diversification (Phase 3)	40
7.7	Automated test suite composition	41

LIST OF FIGURES

5.1	Proposed Framework Architecture	21`
5.2	Data Flow Architecture (Training → Registry → Serving)	22
5.3	Data Flow Diagram (Level 0 / Context Diagram)	23
5.4	Data Flow Diagram (Level 1)	23
5.5	System Workflow Pipeline (single-URL request lifecycle)	24
5.6	Single-URL checker, idle state	24
5.7	SAFE verdict (allowlist stage)	25
5.8	UNSAFE verdict with typosquat note (sbl.co.in vs. sbi.co.in)	25
5.9	INVALID input state	26
5.10	Server-error state (422, over-length URL)	26
5.11	Developer bulk-checking page, idle state	27

1.Preface

1.1 Introduction

The World Wide Web has become the primary channel through which individuals bank, shop, communicate, and access government and healthcare services. This same ubiquity has made the web the most heavily exploited medium for social-engineering attacks, and phishing — the practice of impersonating a trusted entity to trick a victim into revealing credentials, financial information, or installing malware — remains, year after year, one of the top-ranked initial-access techniques in industry threat reports. Unlike malware that exploits a software vulnerability, phishing exploits human trust, and its most common delivery mechanism is deceptively simple: a hyperlink.

Every phishing attack, regardless of whether it arrives by email, SMS ("smishing"), instant messaging, a QR code, or a compromised advertisement, ultimately funnels the victim toward a URL. This makes the URL a uniquely attractive point of interception: if a system can reliably judge, in the few milliseconds before a browser renders a page, whether that URL is attempting to impersonate a legitimate service, an entire class of attacks can be interrupted before any content is even displayed to the user. This is the central premise of the project documented in this report: LexSentinel, a phishing URL detection framework that classifies a URL as safe or unsafe using only the text of the URL string itself.

Restricting the system to URL-text-only analysis is not a limitation adopted for convenience; it is a deliberate architectural decision with concrete engineering benefits. A detector that must fetch and render the destination page to make a decision is slow (a full page load can take seconds), fragile (it can be blocked by the same firewall or CAPTCHA a real user would face, or by a site that behaves differently for a bot than for a human), and privacy-invasive (it must transmit or process the full page content). A detector that reasons purely over the URL string — its length, its character composition, its subdomain structure, the presence of suspicious path keywords, the statistical shape of its host name — can run synchronously inside a single HTTP request, cannot be blocked by a Web Application Firewall (WAF) because it makes no outbound request to the target at all, and is cheap enough to run on every single navigation a browser makes, not merely on a sampled subset.

This report documents the complete lifecycle of that system: the research and dataset landscape that motivated its design, the theoretical foundations of the algorithms used, the system architecture and data-flow design, the concrete technology stack, and — most importantly — the empirical results obtained from training and repeatedly re-evaluating the model against a realistic, deliberately adversarial held-out test set that is disjoint from the training corpus.

A recurring theme throughout this report is the gap between a dataset's own reported test-set metrics and a model's behaviour on traffic that does not share the dataset's collection artefacts — and the concrete engineering process used to close that gap.

1.2 Motivation of the Project

Three converging observations motivated this project.

1.2.1 The scale and persistence of the phishing problem

Phishing has remained, for over a decade, one of the most reported categories of cybercrime to national computer emergency response teams and financial institutions. Anti-Phishing Working Group (APWG) trend reports have repeatedly documented phishing volumes in the hundreds of thousands of unique campaigns per quarter, and financial-services and webmail/SaaS brands remain the most impersonated targets because credential theft against those sectors converts most directly into monetary or further-attack value. Despite this being a well-studied problem, phishing campaigns continue to succeed at scale because attackers can register a new look-alike domain, or a fresh subdomain on a compromised host, faster than static reputation-list defenses can catalogue it — a domain used in a phishing campaign is frequently abandoned within hours of first use, precisely to stay ahead of blocklists.

1.2.2 The limitations of purely list-based and purely content-based defenses

Two dominant families of existing defense exist, and each has a well-documented blind spot. Blocklist/allowlist systems (e.g., Google Safe Browsing, PhishTank, OpenPhish feeds) are extremely precise once a domain is catalogued, but are structurally reactive: a domain must first be reported, verified, and propagated before it is blocked, leaving a "zero-hour" window in which a brand-new phishing domain is invisible to every list-based defense simultaneously. Content- and rendering-based systems (which fetch the page, inspect its DOM, screenshot it, or run a headless browser to compare visual similarity against known brand pages) can catch zero-hour attacks that a list has never seen, but at the cost of latency, server-side compute, and — critically for a defensive tool — exposure to detection and evasion by the attacker's own site (a phishing kit can simply serve different content to a known scanner's user agent or IP range). This project's motivation was to explore a middle path: a lightweight, structural/lexical classifier that needs no page fetch, layered underneath fast deterministic rules for the cases (typosquatting, known brands, known-bad infrastructure) where a rule is more reliable and auditable than a statistical model.

1.2.3 An academic interest in honest, verifiable machine-learning evaluation

A significant part of the motivation behind this project was methodological rather than purely applied: machine-learning coursework and published phishing-detection literature alike frequently report a single held-out test accuracy from the same dataset a model was trained on, without asking whether that dataset's own collection process introduces artefacts a model can exploit without learning the underlying phenomenon at all. This project deliberately built a second, independent evaluation set — URLs the model has never seen, spanning categories the training data does not cover — specifically to test whether the reported metrics generalise. As Chapter 7 documents, the initial model's PhiUSIIL test-set accuracy of 98.3% collapsed to a realistic held-out accuracy of 73.5%, and closing that gap through principled data augmentation (rather than architecture changes) is one of the project's central engineering narratives.

1.3 Problem Statement

Design, implement, and empirically evaluate a phishing URL detection system that:

- Classifies an arbitrary input string as a valid or invalid URL, and — for valid URLs — as safe or unsafe, using only information derivable from the URL text itself, with no network call to the destination host in the decision path.
- Achieves this with sub-second latency per request (and near-linear-time throughput for batches of thousands of URLs), so that it can plausibly sit in front of a live browser navigation without perceptible delay.
- Is resistant to the specific evasion techniques phishing operators are known to use against naive lexical classifiers: brand-name typosquatting (single-character edit distance from a protected domain), homoglyph/Unicode-confusable substitution, leetspeak-style character substitution, and subdomain-prefix abuse (placing a trusted brand name as a subdomain label of an unrelated domain).
- Is evaluated honestly: not merely against the test split of its own training dataset (which, as this report demonstrates, can systematically overstate real-world performance), but against a second, deliberately disjoint set of real legitimate URLs and freshly written phishing-shaped URLs that share no domain, no path template, and no generation logic with anything the model was trained on.
- Is deployable as a complete, usable product — not only a notebook-bound model — including a public web interface, a scriptable bulk-checking API for developers, and a browser extension that protects a user automatically during ordinary browsing.

1.4 Objectives of the Project

The specific, measurable objectives set for this project were:

- Obj. 1 — Acquire and characterise a large, labelled phishing/legitimate URL dataset (the PhiUSIIL Phishing URL Dataset) and identify any collection artefacts in it that could bias a naively trained model.
- Obj. 2 — Design a single, canonical feature-extraction function used identically by both the training pipeline and the live serving API, eliminating train/serve skew as a source of production bugs.
- Obj. 3 — Train and compare at least two families of machine-learning classifier (a linear model, a bagging ensemble, and a boosting ensemble) on the identical feature matrix, and select the best-performing model using metrics computed on both the source dataset's test split and an independent held-out set.
- Obj. 4 — Implement deterministic, explainable defenses (blocklist, allowlist, typosquat/homoglyph detector) as a first line of defense ahead of the statistical model, so that the highest-confidence cases never depend on a black-box score.
- Obj. 5 — Quantify the gap between in-distribution test performance and realistic held-out performance, and close that gap through data-centric engineering (augmentation) rather than model complexity, documenting the before/after evidence.
- Obj. 6 — Expose the system through a production-shaped API (FastAPI), a public web front end, a developer bulk-checking tool, and a Manifest V3 browser extension, and verify the complete request lifecycle end-to-end.
- Obj. 7 — Build an automated regression test suite that encodes every previously confirmed bug and false positive/negative as a permanent, executable test case, so that future changes cannot silently reintroduce a fixed defect.

2. Literature Review

2.1 Review of Existing Research

Automated phishing-URL detection has been studied for roughly two decades, and the research can be read as three overlapping generations: rule-based / heuristic detection, classical machine learning over hand-engineered features, and — most recently — deep representation learning directly over the raw URL string. This project draws primarily on the second generation, for reasons discussed at the end of this section.

2.1.1 Rule-based and heuristic detection

Early anti-phishing research focused on heuristics that a human analyst could articulate directly: does the URL contain an "@" symbol (historically used to smuggle a real destination past a displayed fake host), does it use an IP address literal instead of a domain name, is the domain age (via WHOIS) suspiciously recent, does the page's visual layout closely resemble a known bank's login page while being hosted on an unrelated domain. Zhang, Hong and Cranor's CANTINA (2007) is a representative system from this generation: it combined term-frequency/inverse-document-frequency (TF-IDF) analysis of a page's own text with search-engine cross-checks (a legitimate site should rank for its own most distinctive terms; a cloned phishing page usually does not) and a small set of hand-written heuristic rules. These systems are highly interpretable and can be extremely precise for the exact patterns they encode, but they generalise poorly: every new evasion trick (a novel obfuscation, a newly abused free hosting provider, a previously unseen brand target) requires a human to notice it and hand-author a new rule, which does not scale against an adversary that iterates faster than any single team of analysts.

2.1.2 Classical machine learning over hand-engineered URL/lexical features

The dominant paradigm in the academic literature — and the one this project builds on most directly — replaces hand-authored rules with a classifier trained on a vector of engineered features: URL length, number of dots/hyphens/digits, presence of an IP-address host, domain entropy, subdomain count, presence of "suspicious" keywords such as "login" or "secure" in the path, and similar lexical/structural statistics. Sahingoz, Buber, Demir and Dirir (2019) is a widely cited example: the authors extracted purely lexical, NLP-style, and word-vector features from raw URLs (deliberately avoiding third-party services such as WHOIS or page rank, for the same latency/availability reasons this project avoids them) and showed that Random Forest classifiers over these features achieved accuracy above 97% on their evaluation corpus, outperforming several

other classical learners they compared against. This body of work established two points this project relies on directly: (a) URL-string-only features carry enough signal for high-accuracy classification without needing page content, and (b) ensemble tree methods (Random Forest, and later gradient-boosted trees) consistently outperform simpler linear models on this feature family, because the decision boundary between legitimate and phishing URLs is highly non-linear across feature interactions (e.g. "many hyphens" is only suspicious in combination with an uncommon TLD and a credential-harvesting keyword, not in isolation).

2.1.3 Deep representation learning over the raw URL string

A more recent line of work removes feature engineering entirely and learns a representation directly from the URL's raw characters. Le, Pham, Sahoo and Hoi's URLNet (2018) is the canonical example: a convolutional neural network operating in parallel over character-level and word-level embeddings of the URL, trained end-to-end for malicious-URL classification. Such models can discover feature interactions no human would think to hand-encode, and character-level convolution in particular is naturally robust to the exact character-substitution obfuscation (leetspeak, homoglyphs) this project's typosquat/wordplay module handles with explicit rules. Their principal costs are training-data volume (deep models need substantially more labelled examples per parameter to avoid overfitting than a gradient-boosted tree over a curated feature set), inference cost (a forward pass through a multi-branch CNN is markedly more expensive per URL than evaluating a few hundred shallow decision trees), and interpretability (a learned embedding space does not decompose into an auditable "this URL was flagged because of X" the way a tree ensemble's feature importances or a rule engine's matched rule does).

2.1.4 The dataset landscape, and why PhiUSIIL

Reported accuracy in this literature is only as trustworthy as the dataset it is measured against. Early academic datasets (e.g. the widely used UCI "Phishing Websites" dataset) are now over a decade old and reflect an obsolete threat landscape, are small by modern standards (a few thousand rows), and in several cases provide only pre-computed feature vectors rather than raw URLs — making it impossible to verify how a feature was actually derived or to re-engineer new ones. This project uses the PhiUSIIL Phishing URL Dataset (Prasad and Chandra, 2023/2024), a substantially larger and more recent corpus of 235,795 labelled URLs (134,850 legitimate, 100,945 phishing) that ships both the raw URL string and a rich set of the original authors' own precomputed URL- and webpage-content-derived features. Critically for this project, having the raw URL string available means feature engineering does not have to trust or depend on the dataset authors' own precomputed columns — every feature used in this project is re-derived from the raw URL text by this project's own

code (Chapter 3, Chapter 5), which is what makes it possible to compute the identical feature set at serving time, when only the raw URL (never the dataset's precomputed columns, and never the destination page's content) is available.

2.2 Existing Systems and Limitations

Beyond the academic literature, several production systems already defend the average user against phishing URLs. Understanding their design and limitations directly shaped this project's own architecture, particularly the decision to layer a statistical model underneath deterministic rules rather than relying on either alone.

2.2.1 Community and vendor blocklists (PhishTank, OpenPhish, URLhaus)

PhishTank and OpenPhish aggregate phishing URLs reported by users and security researchers into a continuously updated feed; URLhaus does the same for malware-distribution URLs. These feeds are the backbone of this project's own blocklist stage design (core/lists.py's seed blocklist is explicitly documented as a placeholder for exactly such a feed in production). Their strength is precision — a URL on the list has been human- or community-verified — but their fundamental limitation is latency-to-coverage: a URL must be seen, reported, and verified before it appears on the list, during which every user is unprotected. Because phishing domains are frequently used for only hours before abandonment, a purely blocklist-driven defense is structurally always a step behind.

2.2.2 Browser- and OS-integrated reputation systems (Google Safe Browsing, Microsoft SmartScreen)

Google Safe Browsing and Microsoft Defender SmartScreen operate at much greater scale than any community list, combining crawler-discovered URLs, telemetry from hundreds of millions of endpoints, and (in SmartScreen's case) application-reputation signals. They are the most effective large-scale defenses in general deployment today. Their limitation, from the perspective of this project, is that they are closed, centralised systems: the exact features and models are not public, they cannot be run offline or self-hosted, and — most relevant to this project's academic purpose — they cannot be studied, retrained, or extended by a student or a small organisation. They also share the blocklist family's fundamental reactive character for genuinely novel domains, mitigated only by the sheer scale of their crawling infrastructure.

2.2.3 Content-rendering and visual-similarity detectors

A separate class of commercial and research tool renders the destination page (often in a sandboxed headless browser) and compares its visual layout, logo

placement, and DOM structure against a reference set of legitimate brand pages. These can catch a zero-hour clone of a well-known login page with high precision, but require fetching and executing the target page for every check — the exact cost, latency, and evasion surface (a phishing kit can cloak its content from known scanner IP ranges or user agents) this project's design deliberately avoids by never making an outbound request to the URL being checked.

2.2.4 Browser extensions built on third-party reputation APIs

Numerous browser extensions exist that simply call a third-party reputation API (often one of the systems above) per navigation. Functionally, this is close to what this project's own Manifest V3 extension does — intercept navigation, check the URL, redirect if unsafe — but such extensions add no detection capability of their own: they are only as good, and only as available, as the API they depend on, and they typically cannot explain why a URL was flagged beyond a generic warning. This project's extension instead talks to a self-hosted, inspectable backend running an auditable, retrainable model plus explicit rule stages, each surfaced to the user with a specific stage ("blocklist", "typosquat", "model") and, where applicable, a note explaining the exact resemblance detected.

2.3 Comparative Analysis Table

Table 2.1 summarises the preceding discussion across the dimensions most relevant to a phishing-URL defense: detection basis, ability to catch a never-before-seen ("zero-hour") domain, latency/offline capability, transparency of a given verdict, and typical deployment form. LexSentinel is included in the final row for direct comparison.

Table 2.1 — Comparative analysis of phishing-detection approaches

Approach	Detection basis	Zero-hour capable?	Latency / offline	Verdict transparency	Typical deployment
Community blocklists (PhishTank, OpenPhish)	Human/community-verified URL match	No — reactive by design	Very low latency; needs periodic feed sync	High (exact match)	Feed consumed by other tools
Browser-vendor reputation (Safe Browsing, SmartScreen)	Proprietary, large-scale crawling + telemetry	Partial — scale mitigates lag	Low latency; requires network/vendor service	Low (opaque)	Built into browser / OS
Visual / content-similarity detectors	Rendered page vs. brand reference images	Yes, for cloned login pages	High latency; requires fetching target page	Medium (similarity score)	Server-side scanning pipeline
Classical ML on hand-engineered features (Sahingoz et al., 2019)	Lexical/structural URL features + classifier	Yes, if features generalise	Low latency; fully offline-capable	Medium (feature importances)	Research prototype / embedded classifier
Deep URL representation learning (URLNet, 2018)	Character/word CNN embeddings	Yes, if trained on diverse data	Moderate latency; offline-capable	Low (learned embedding)	Research prototype
LexSentinel (this project)	Rules (block/allow/typosquat) + XGBoost on lexical features + char n-gram TF-IDF	Yes for the ML stage; rules catch known brands	Low latency; fully offline-capable, no network call to target	High — verdict tagged with stage + note	Self-hosted API, web UI, browser extension

The comparison makes explicit the gap this project set out to fill: a system that has the offline, low-latency, zero-hour-capable profile of a classical-ML lexical classifier, but adds the deterministic precision and explainability of a rule-based system for the specific, high-value cases (known brands, known-bad infrastructure, typosquatting) where a rule is more trustworthy than a probability score — and, unlike a closed vendor system, is fully inspectable, retrainable, and self-hostable.

3. Dataset Analysis And Requirements Study

3.1 Functional Requirements

Functional requirements describe what the system must do, independent of how well it does it. They were derived directly from the intended end-to-end use of LexSentinel across its three client surfaces (web UI, browser extension, developer bulk-checking tool).

Table 3.1 — Functional requirements

ID	Requirement
FR-1	Accept a single URL (typed or pasted, with or without a scheme) and return a verdict of safe, unsafe, or invalid, via both a web page and a JSON API.
FR-2	Validate that the submitted input is a plausible URL (has a hostname with a dot and a real-looking TLD, or is a literal IP address) before any classification is attempted, and return a user-facing explanation when it is not.
FR-3	Check every classified URL against a blocklist and an allowlist before invoking the machine-learning model, and short-circuit with an immediate verdict when either matches.
FR-4	Detect brand-typosquatting and subdomain-prefix impersonation of a curated set of protected domains, independent of the ML model's own judgement.
FR-5	Return, for every verdict, the exact stage that produced it (blocklist / allowlist / typosquat / model), a confidence score when the model stage is used, and the active model's version identifier.
FR-6	Accept a batch of up to 5,000 URLs in one request (as JSON or as an uploaded .txt/.csv file) and return per-URL verdicts plus an aggregate summary, gated behind a developer authentication key.
FR-7	Export bulk-check results as a downloadable CSV file, safe against CSV-injection when opened in a spreadsheet application.
FR-8	Automatically evaluate every site a user's browser navigates to (via the companion browser extension) and interstitially warn before the destination page is allowed to render, without requiring any manual user action.
FR-9	Allow a developer to hot-reload the active model and the allow/blocklists without restarting the running server process.
FR-10	Expose a health-check endpoint reporting service status and the currently loaded model's version, for deployment monitoring.
FR-11	Provide a reproducible training pipeline that reads the labelled dataset, extracts features via a single canonical function, trains a classifier, and writes a versioned, independently loadable model artifact plus its metadata.

3.2 Non-Functional Requirements

Table 3.2 — Non-functional requirements

Category	Requirement
Performance	A single-URL check must complete well under one second end-to-end; a 5,000-URL bulk request must complete in well under a second of pure model-inference time by batching feature extraction and prediction rather than looping per URL.
Security	Bulk-checking and administrative endpoints must require an authenticated, auto-generated developer key; rate-limiting must key off the platform-appended (last) X-Forwarded-For entry, never a client-supplied one; uploaded-file size and per-item URL length must be capped to prevent denial-of-service; CSV exports must neutralise spreadsheet-formula injection.
Privacy	Server logs must record only the domain of a checked URL, never its full path or query string, since those can carry search terms, session tokens, or other user-identifying data.
Reliability	A missing or corrupted model artifact must degrade to an explicit 503 Service Unavailable response, never an unhandled 500 error; a single malformed URL in a bulk batch must not abort the rest of the batch.
Portability	The trained model must load and serve identically on the operating system it was trained on and on the (possibly different) operating system it is deployed to, without a cross-platform serialization failure.
Maintainability	Feature-extraction logic must exist in exactly one place, imported identically by the training pipeline and the serving layer, so the two paths can never silently diverge.
Auditability	Every verdict must be traceable to the exact decision stage that produced it, and the exact feature-set version and constants (keyword lists, TLD lists) used to produce a given model must be recorded in that model's metadata.
Usability	The web interface must present a clear, unambiguous visual state for each of the four possible outcomes (safe, unsafe, invalid input, server error), and must never display a stale result from a previous, superseded request.
Extensibility	Allowlist, blocklist, and protected-brand data must live in external, editable configuration rather than hardcoded in source, so operational lists can be refreshed without a code change.

3.3 Dataset Overview

The model in this project is trained on the PhiUSIIL Phishing URL Dataset (Prasad and Chandra, 2023/2024), a large, recent, and — unlike several older academic phishing datasets — raw-URL-preserving corpus published on Kaggle and mirrored on the UCI Machine Learning Repository. Each row represents one URL, carrying both the original URL string and 54 additional columns describing lexical, structural, and webpage-content properties, plus a binary label.

Table 3.3 — PhiUSIIL dataset composition (as verified on the locally loaded copy)

Property	Value
Total rows	235,795
Legitimate URLs (label = 1)	134,850 (57.2%)
Phishing URLs (label = 0)	100,945 (42.8%)
Raw columns supplied	55 (including FILENAME, URL, Domain, TLD, 51 lexical/structural/content features, and label)
Format	Single CSV file, $\approx$ 56 MB
Label convention used by this project	Inverted at load time: 1 = phishing, 0 = legitimate (see Section 3.4)

Two properties of this dataset were empirically verified during requirement analysis and directly shaped the feature-engineering and augmentation strategy described in Chapters 5 and 7:

- The label column is inverted relative to the intuitive convention: label = 1 denotes a legitimate URL and label = 0 denotes phishing in the raw file, confirmed against known real URLs (e.g. a university domain mapped to label = 1; a throwaway free-TLD domain mapped to label = 0). This project flips the label at load time ($y = 1 - \text{raw_label}$) so that 1 = phishing consistently throughout the rest of the codebase.
- The legitimate class is drawn overwhelmingly from bare homepage URLs: essentially 100% of legitimate rows are a domain root with no meaningful path, while a majority of phishing rows carry a path (a credential-harvesting form location such as /login.php). A model trained on this distribution unmodified learns "any URL with a real path is suspicious" — a shortcut that fails on ordinary web traffic (articles, product pages, documentation). This artefact, and the training-data augmentation strategy built to counteract it, is discussed in Sections 5.3–5.4 and evaluated empirically in Chapter 7.

3.4 Attribute Description

3.4.1 Attributes supplied by the raw dataset

The 55 raw columns fall into four broad groups. This project uses only the raw URL string and the label; every other supplied column is deliberately discarded (Section 5.3 explains why), but they are catalogued here because they define the space of information a page-content-based competitor system (Section 2.2.3) would additionally depend on, and because their presence in the dataset is itself part of what a requirement analysis of this corpus must document.

Table 3.4 — Attribute groups supplied by the PhiUSIIL dataset

Group	Representative columns	Used by this project?
Identity / meta	FILENAME, URL, Domain, TLD, label	URL and label only
URL-lexical (string-derivable)	URLLength, DomainLength, NoOfSubDomain, IsDomainIP, IsHTTPS, NoOfLettersInURL, NoOfDgitsInURL, NoOfEqualsInURL, NoOfQMarkInURL, NoOfAmpersandInURL, NoOfObfuscatedChar, ObfuscationRatio	No — re-derived independently (see 3.4.2 / Chapter 5)
Dataset-authors' own similarity features	URLSimilarityIndex, CharContinuationRate, TLDLegitimateProb, URLCharProb	No — this project defines its own feature set
Webpage-content / DOM-derived (requires fetching the live page)	LineOfCode, HasTitle, HasFavicon, IsResponsive, NoOfURLRedirect, HasExternalFormSubmit, HasSocialNet, HasSubmitButton, HasHiddenFields, HasPasswordField, Bank, Pay, Crypto, NoOfImage, NoOfCSS, NoOfJS, NoOfIFrame, NoOfPopup	No — excluded by design (no network call in the request path; see 5.3)

The exclusion of the content/DOM-derived group is a deliberate architectural boundary, not an oversight: obtaining any of those values requires fetching and often rendering the destination page at check time, which reintroduces exactly the latency, availability, and evasion problems Section 1.2.2 and Section 2.2.3 identify as the core weakness of content-based detectors. Similarly, the dataset authors' own similarity-index features (their paper's central contribution) are not reused directly, both to keep this project's feature semantics fully self-defined and auditable, and because those columns are precomputed only for this dataset's rows — they cannot be recomputed for an arbitrary incoming URL at serving time without reimplementing the original authors' own (separately published) similarity-index algorithm end-to-end.

3.4.2 Attributes engineered by this project (core/features.py)

Because none of the dataset's own precomputed columns can be trusted to reproduce identically at serving time (Section 5.3), this project defines its own 34 numeric features plus one text field, computed by a single function — `extract_features()` — shared, unmodified, between training and serving. Table 3.5 documents every feature.

Table 3.5 — The 34 engineered numeric features (+ 1 text field) used by the model

#	Feature	Description
1	url_length	Character length of the URL (bare-root URLs normalised; see 5.3)
2	domain_length	Character length of the hostname (www.-stripped)
3	path_length	Character length of the URL path
4	query_length	Character length of the query string
5	num_dots	Count of '.' characters
6	num_hyphens	Count of '-' characters

#	Feature	Description
7	num_underscores	Count of '_' characters
8	num_slashes	Count of '/' characters
9	num_digits	Count of digit characters
10	num_letters	Count of alphabetic characters
11	digit_ratio	num_digits / url_length
12	letter_ratio	num_letters / url_length
13	num_special_chars	Count of non-alphanumeric characters
14	special_char_ratio	num_special_chars / url_length
15	num_subdomains	Host labels before the registrable domain + TLD
16	is_ip_address	1 if the host is a literal IPv4/IPv6 address
17	is_https	1 if the scheme is https
18	tld_length	Character length of the top-level domain label
19	num_equals	Count of '=' characters
20	num_question_marks	Count of '?' characters
21	num_ampersands	Count of '&' characters
22	num_percent_encoded	Count of %XX percent-encoded sequences
23	has_obfuscation	1 if any percent-encoding is present
24	domain_entropy	Shannon entropy of the hostname string
25	has_at_symbol	1 if '@' appears in the URL
26	num_path_segments	Count of non-empty '/'-separated path segments
27	suspicious_keyword_count	Path/query matches against a curated keyword list (login, verify, secure, banking, ...), leetspeak-aware
28	has_port	1 if an explicit, valid port is present
29	is_common_tld	1 if the TLD is in a small curated common-TLD set
30	has_mixed_script	1 if the hostname mixes Latin with Cyrillic/Greek characters (homograph signal)
31	is_punycode	1 if any host label is punycode-encoded (xn--...)
32	num_confusable_chars	Count of leetspeak/homoglyph substitution characters in the hostname
33	confusable_char_ratio	num_confusable_chars / domain_length
34	domain_has_obfuscated_suspicious_term	1 if the hostname, only after confusable-normalisation, reveals a suspicious term (e.g. 's3cure')

Every feature above is computed exclusively from the URL string, with no network call, satisfying the offline/WAF-safety requirement (Section 1.3) and guaranteeing that identical input always yields identical features, whether computed during training on a 235,795-row batch or during a live single-URL request.

4. Related Theories and Algorithms

4.1 Fundamental Theories Underlying the Work

4.1.1 Binary classification and the bias–variance trade-off

At its core, phishing-URL detection is framed in this project as a supervised binary classification problem: given a feature vector x derived from a URL, learn a function $f(x) \rightarrow [0, 1]$ estimating the probability that the URL is phishing, then threshold that probability at an operating point (Section 4.1.6) to produce a discrete safe/unsafe verdict. Every model considered in this project is trained by minimising a loss function (log-loss for the linear and boosted-tree models) over a labelled training set, subject to the classical bias–variance trade-off: a model too simple for the true decision boundary (high bias) underfits and misses genuine phishing patterns, while a model with excess capacity relative to the training data (high variance) memorises training-specific artefacts rather than learning a generalisable rule. Chapter 5's discussion of training-data augmentation, and Chapter 7's documented case of a model memorising specific augmented training URLs rather than generalising their shape, are both direct, empirically observed instances of the variance side of this trade-off.

4.1.2 Ensemble learning: bagging versus boosting

Both alternative classifiers evaluated in this project against the final choice (Chapter 7) are ensembles — combinations of many simple models whose aggregate prediction is more accurate and more stable than any single member. The two ensembling strategies compared are theoretically distinct. Bagging (bootstrap aggregating), the basis of Random Forest, trains many decision trees independently, each on a different bootstrap resample of the training data and a random subset of features at each split, then averages their predictions; because the trees are trained independently and in parallel, bagging primarily reduces variance without much affecting bias. Boosting, the basis of XGBoost, instead trains trees sequentially, where each new tree is fit specifically to the residual error the ensemble-so-far still makes, so the ensemble progressively reduces bias (and, with appropriate regularisation, variance too). This distinction is why the literature (Section 2.1.2) and this project's own empirical comparison (Section 7.7) both find boosted trees edging out bagged trees on this exact feature family: boosting's sequential error-correction is better suited to a decision boundary shaped by many specific, conditionally-interacting feature combinations (Section 4.1.1), which is characteristic of adversarially designed phishing URLs.

4.1.3 Information theory: Shannon entropy as a randomness signal

One engineered feature (`domain_entropy`, Table 3.5) is drawn directly from Claude Shannon's information theory. For a hostname string composed of characters with observed frequencies $p_1 \dots p_k$, the Shannon entropy is:

$$H(\text{domain}) = - \sum p_i \cdot \log_2(p_i) \quad \text{for each distinct character } i \text{ in the hostname}$$

A hostname assembled from a pronounceable brand name ("paypal", "microsoft") has comparatively low entropy — a small alphabet repeated in patterned ways — whereas an algorithmically generated or randomly padded hostname (as is common in fast-flux and domain-generation-algorithm-driven phishing infrastructure) tends toward higher, more uniform character distribution and therefore higher entropy. This feature does not decide anything by itself; it contributes one axis of evidence the tree ensemble can combine with others.

4.1.4 Text representation theory: bag-of-character-n-grams and TF-IDF

Alongside the 34 numeric features, the model consumes the URL's path and query string through a Term Frequency–Inverse Document Frequency (TF-IDF) representation over character n-grams (contiguous substrings of length 3 to 5). For a term (character n-gram) t in a document (one URL's path+query text) d , drawn from a corpus D :

$$\begin{aligned} \text{tf}(t, d) &= \text{number of occurrences of } t \text{ in } d \\ \text{idf}(t, D) &= \log\left(\frac{|D|}{|\{d \text{ in } D : t \text{ in } d\}|} \right) \\ \text{tfidf}(t, d, D) &= \text{tf}(t, d) \times \text{idf}(t, D) \end{aligned}$$

The deliberate choice of character n-grams (rather than whole-word tokens) over the path and query alone (never the hostname — Table 3.5's suspicious-keyword rationale applies equally here) is what gives this representation practical robustness to the exact obfuscation techniques phishing URLs use: a substring such as "log" or "ver" still appears inside a leetspeak variant like "l0gin" or "ver1fy" at the character level even though no whole-word tokenizer would match it. The inverse-document-frequency term additionally down-weights n-grams that are common across almost every URL (low discriminative value) in favour of n-grams distinctive to one class or the other.

4.1.5 Edit-distance theory and Unicode confusables

Typosquatting detection rests on string-metric theory: the minimum number of elementary edit operations required to transform one string into another. Classical Levenshtein distance permits insertion, deletion, and substitution; Damerau–Levenshtein distance (Section 4.2.5) additionally permits an adjacent-character transposition as a single edit, which matters because keyboard-adjacent typos and deliberate brand-name scrambles ("flipkart" → "filpkart")

are overwhelmingly transposition errors, and plain Levenshtein distance would otherwise count them as two edits, pushing them past a distance-1 threshold undetected. Complementing edit distance, Unicode confusable theory recognises that many characters from different scripts are visually near-identical to a human reader (Cyrillic "a" U+0430 versus Latin "a" U+0061) while being distinct at the code-point level a naive string comparison operates on — the basis of the classic IDN homograph attack, and the reason this project normalises such characters to their canonical Latin form before any distance or keyword comparison is performed.

4.1.6 Decision thresholds and the precision–recall trade-off

A classifier's probability output must be converted to a discrete verdict by choosing an operating threshold. For a security product, this choice encodes a deliberate cost trade-off between two asymmetric error types: a false positive (a legitimate site wrongly blocked) causes user friction and erodes trust in the tool, while a false negative (a phishing site wrongly allowed) causes direct harm to the user. This project's serving layer uses a single, centrally defined threshold (`DECISION_THRESHOLD = 0.5`, Section 5.6) applied identically wherever a probability is produced, and documents — rather than hides — the fact that 0.5 is a naive default rather than a value tuned against a precision/recall curve, leaving threshold tuning as explicit, tracked future work (Chapter 8).

4.2 Fundamental Algorithms

4.2.1 Decision trees (CART)

Both ensemble methods used in this project (Random Forest and XGBoost) are built from Classification and Regression Trees (CART). A decision tree recursively partitions the feature space by repeatedly choosing the single feature and threshold that most reduces impurity in the resulting two child partitions, typically measured by Gini impurity for classification:

$$\text{Gini}(\text{node}) = 1 - \sum p_c^2 \quad \text{for each class } c \text{ present in that node}$$

Recursion continues until a stopping criterion (maximum depth, minimum samples per leaf) is met. A single tree is easy to interpret but prone to overfitting; both ensembling strategies below exist specifically to address that weakness.

4.2.2 Random Forest

Random Forest (Breiman, 2001) grows many CART trees, each on an independent bootstrap sample of the training rows and, at every split, a random subset of the available features (rather than the single best feature over the full set). This double randomisation decorrelates the individual trees, so that averaging their predictions (or, for classification, majority-voting or averaging

class probabilities) cancels out much of each tree's individual variance. This project's comparison model (Section 7.7) uses 200 trees with balanced class weighting.

4.2.3 Gradient boosting and XGBoost

Gradient boosting builds an ensemble additively: starting from an initial prediction, each successive tree is fit not to the original labels but to the gradient of the loss function with respect to the current ensemble's predictions — informally, each new tree is trained to correct the specific mistakes the ensemble-so-far is still making. XGBoost (Extreme Gradient Boosting; Chen and Guestrin, 2016), the classifier this project ultimately deploys (Section 5.5, Chapter 7), refines this idea with a regularised learning objective that penalises tree complexity directly, a second-order (Newton-method) Taylor expansion of the loss for more accurate per-split gain estimates, and engineering optimisations (column subsampling, weighted quantile sketching for split-finding, sparsity-aware handling of missing values, and multi-threaded histogram building) that make it dramatically faster to train than a naive gradient-boosting implementation at the same accuracy. Its per-iteration objective for a tree structure q with T leaves and leaf weights w is:

$$\text{Obj}(t) = \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \frac{\Omega(f_t)}{\gamma \cdot T + \frac{1}{2} \lambda \cdot \sum_j w_j^2}$$

where g_i , h_i are the first- and second-order gradients of the loss for row i , and γ , λ are complexity-penalty hyperparameters.

This project's trained model uses 400 trees, a maximum depth of 6, a learning rate (shrinkage) of 0.1, and 90% row/column subsampling per tree (train.py), with `scale_pos_weight` set to the benign:phishing ratio of the augmented training set to counteract residual class imbalance without discarding data.

4.2.4 Logistic Regression (baseline comparison model)

Logistic Regression models the log-odds of the positive class as a linear combination of the input features, passed through the logistic (sigmoid) function:

$$P(\text{phishing} | x) = 1 / (1 + e^{-(w \cdot x + b)})$$

It is trained by minimising log-loss (cross-entropy) via gradient-based optimisation, typically with an L2 penalty on the weight vector to control overfitting. Its central limitation for this task, borne out empirically in Section 7.7, is that it can only represent a linear decision boundary over the input features (or the TF-IDF space); it cannot natively express the conditional feature interactions (Section 4.1.2) that meaningfully separate phishing from legitimate URLs in this feature space.

4.2.5 Damerau–Levenshtein edit distance

The typosquat detector's core primitive is a dynamic-programming computation of edit distance between a candidate domain "core" and a protected brand's core (Section 5.x, `core/typosquat.py`). For strings *a* (length *m*) and *b* (length *n*), the recurrence extends classical Levenshtein distance with a transposition case:

```
d[i][j] = min(
    d[i-1][j] + 1,           # deletion
    d[i][j-1] + 1,           # insertion
    d[i-1][j-1] + cost(a_i, b_j), # substitution (cost 0 if a_i == b_j)
    d[i-2][j-2] + 1 if a_i==b_(j-1) and a_(i-1)==b_j # transposition
)
```

This project runs both the Damerau–Levenshtein variant and a plain-Levenshtein variant side by side (`core/typosquat.py`'s `_has_corroborating_signal` caller) specifically to detect whether a given distance-2 match was reached via a transposition — a much stronger deliberate-typosquat signal than two coincidental single-character substitutions — and applies a stricter corroboration requirement (a suspicious path keyword or an unusual TLD) only to the latter, weaker case.

4.2.6 Unicode NFKC normalisation

To close gaps in a hand-maintained homoglyph table, hostnames are additionally passed through Unicode Normalization Form KC (NFKC) compatibility normalisation before comparison, which folds compatibility-equivalent characters (for example, fullwidth Latin forms such as "a—"z", common in some East-Asian keyboard layouts and occasionally seen in obfuscated phishing domains) to their canonical ASCII forms, at effectively no implementation cost beyond calling the standard library's `unicodedata.normalize`.

4.3 Summary

Table 4.1 maps each theory or algorithm above to the specific place it is used in the system, previewed here ahead of the full architectural discussion in Chapter 5.

Table 4.1 — Theory-to-implementation mapping

Theory / algorithm	Used in
Binary classification / bias-variance	Overall model training objective (models/train.py)
Bagging (Random Forest)	Baseline comparison classifier (Section 7.7)
Gradient boosting (XGBoost)	Deployed production classifier
Shannon entropy	domain_entropy feature (core/features.py)
TF-IDF over character n-grams	Path/query text branch of the model pipeline
Damerau–Levenshtein distance	Typosquat detection (core/typosquat.py)
Unicode confusables / NFKC	Homoglyph and leetspeak detection (core/wordplay.py)
Precision–recall trade-off	DECISION_THRESHOLD design (core/registry.py)

5. Proposed Framework and Methodology

5.1.1 Proposed Framework Architecture

LexSentinel is organised as three layers: a client layer (the web UI, the browser extension, and any script or tool calling the bulk API), an API/serving layer (a FastAPI application exposing single-check, bulk-check, health, and admin-reload endpoints), and a decision pipeline that implements defense in depth — a fast deterministic stage (blocklist, allowlist, typosquat/wordplay detection) checked before the statistical machine-learning stage is ever invoked. Figure 5.1 shows the complete layered architecture.

Figure 5.1 — Proposed Framework Architecture

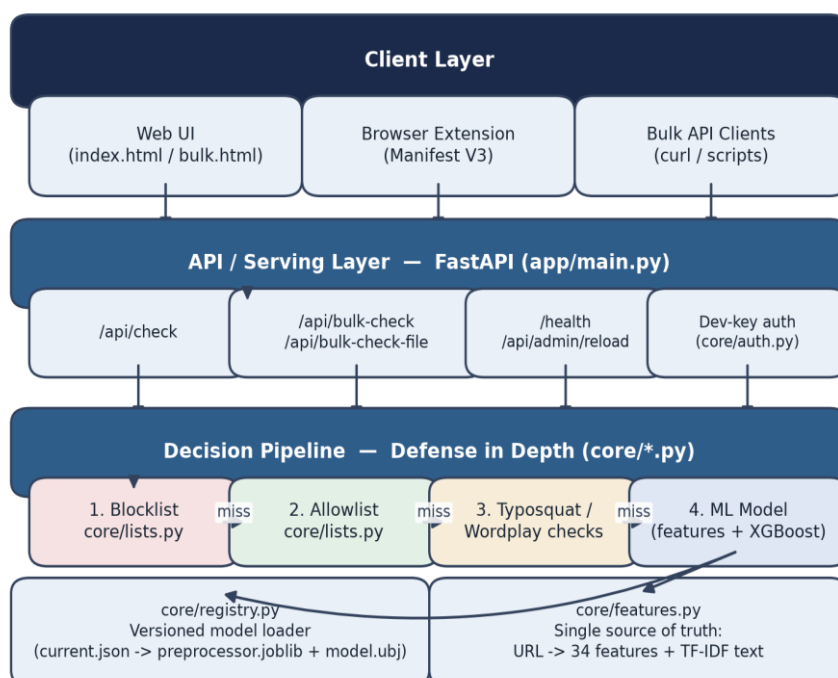


Figure 5.1 — Proposed Framework Architecture

The ordering of the decision pipeline (blocklist → allowlist → typosquat → model) is itself a design decision, not an arbitrary sequence: a blocklist hit is the cheapest and most certain signal available and is checked first; an allowlist hit is checked next because a known-good domain should never be second-guessed by a downstream, occasionally-wrong statistical model; typosquat/wordplay detection runs third because it is a high-precision, explainable rule better trusted than a bare probability score for the specific case of brand impersonation; only a

URL that clears all three deterministic checks reaches the ML model, which acts as a general-purpose fallback for patterns no explicit rule anticipated.

5.1.2 Data Flow Architecture

Figure 5.2 traces how data moves from the raw training dataset through to a live prediction, spanning both the offline training pipeline (models/train.py) and the online serving path (app/main.py). The two paths share exactly one component — core/features.py's extract_features() — which is the architectural guarantee that training and serving can never silently disagree about what a given URL's feature vector should be.

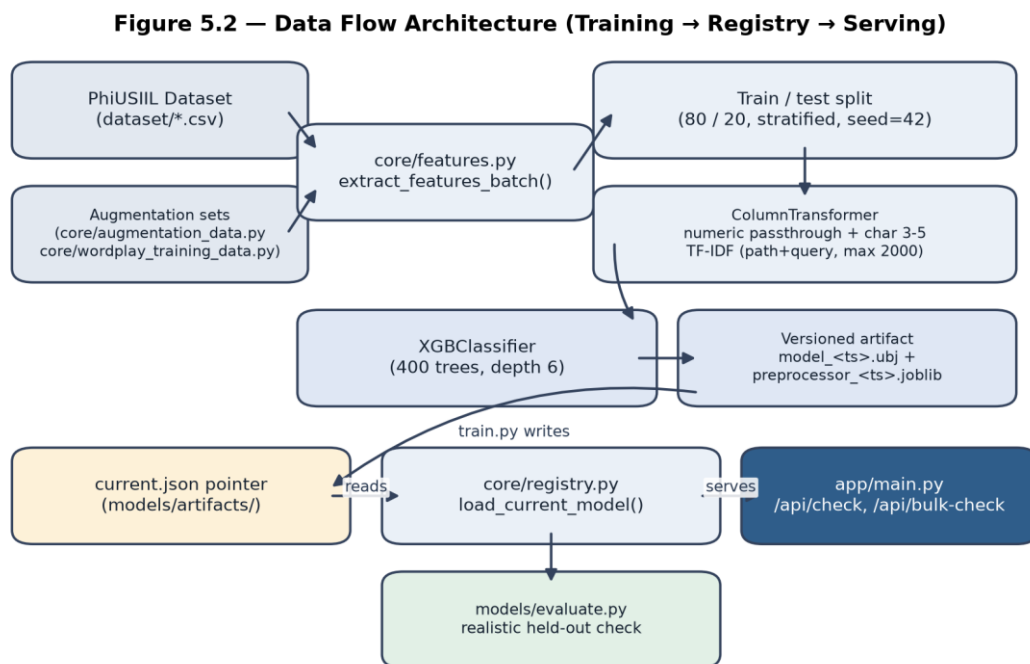


Figure 5.2 — Data Flow Architecture (Training → Registry → Serving)

5.1.3 Data Flow Diagram (Level 0)

The Level 0 (context) diagram treats the entire system as a single process and identifies its external entities: the end user submitting a URL for checking, the developer/administrator performing bulk checks or triggering retraining, and the two offline data sources (the config maintainer who refreshes allow/blocklists, and the PhiUSIIL dataset used for training).

Figure 5.3 — Data Flow Diagram (Level 0 / Context Diagram)

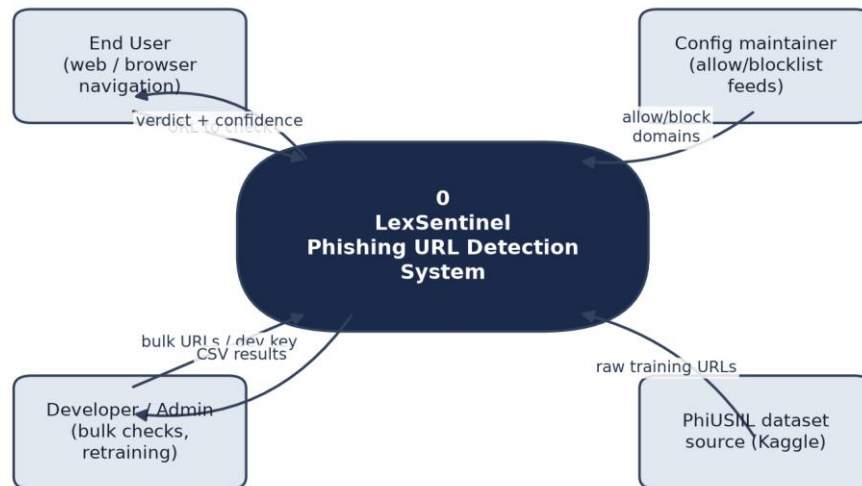


Figure 5.3 — Data Flow Diagram (Level 0 / Context Diagram)

5.1.4 Data Flow Diagram (Level 1)

Decomposing the single Level 0 process into its constituent functions produces Figure 5.4: URL validation, the three-stage deterministic rule check, feature extraction, ML classification, and response assembly/logging, each reading from or writing to one of four data stores (the allow/blocklist configuration, the protected-brand reference set reused by the typosquat detector, the versioned model artifact, and the domain-only verdict log).

Figure 5.4 — Data Flow Diagram (Level 1)

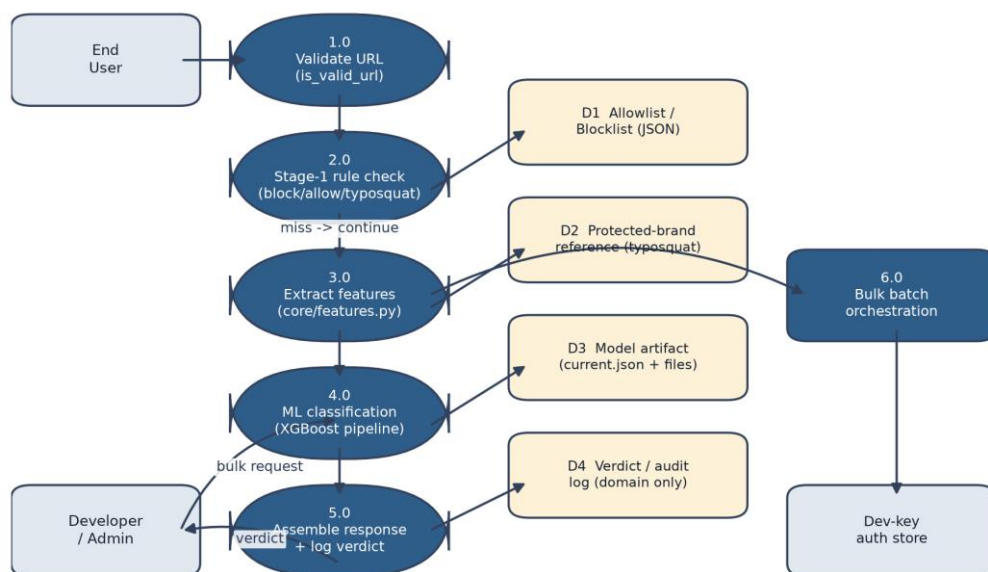


Figure 5.4 — Data Flow Diagram (Level 1)

5.2.1 System Workflow Pipeline

Figure 5.5 traces a single request end to end as it is actually processed by `app/main.py`'s `check_url()` function: length/format validation first (invalid input never reaches any classifier), then the three deterministic stages in order (each an early-exit point that returns a verdict without ever invoking the model), and finally, only for URLs that fall through all three, feature extraction, XGBoost probability estimation, and threshold-based verdict assignment.

Figure 5.5 — System Workflow Pipeline (single-URL request lifecycle)

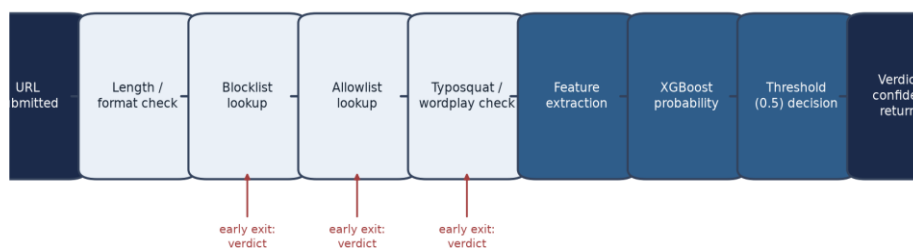


Figure 5.5 — System Workflow Pipeline (single-URL request lifecycle)

5.2.2 User Interface Snapshots

The following figures are genuine screenshots of the running application (captured from a live local instance of `app/main.py`, driving the exact HTML/CSS/JS shipped in `app/static/`), not mock-ups. Figure 5.6 shows the public single-URL checker's idle state before any check has been submitted.

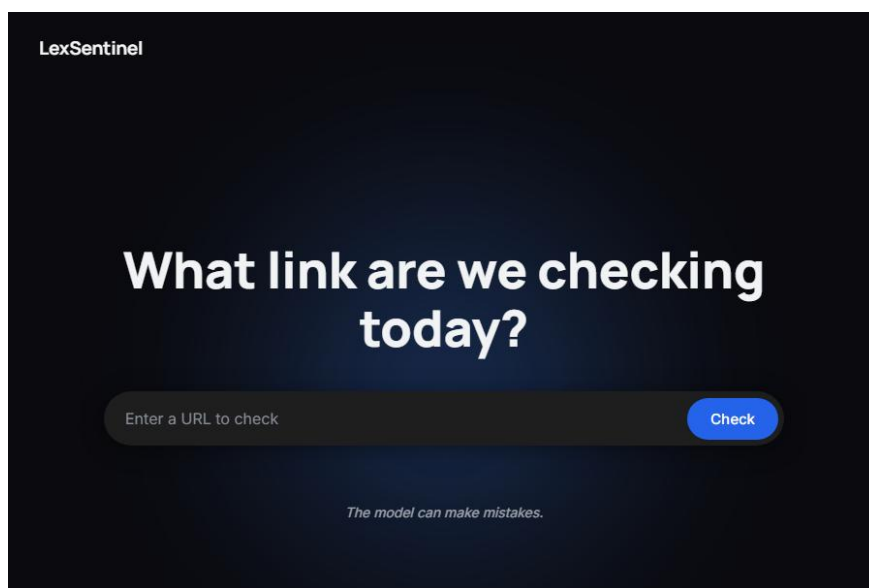


Figure 5.6 — Single-URL checker, idle state

Figure 5.7 shows the SAFE state (green glow, green badge) for a legitimate URL recognised by the allowlist stage. Figure 5.8 shows the UNSAFE state (red glow, red badge) for `http://sbl.co.in/` — the exact example cited in this project's own typosquat-detection module and extension documentation, one Damerau–Levenshtein edit away from the protected domain `sbi.co.in`, correctly flagged with an explanatory note rather than a bare verdict.

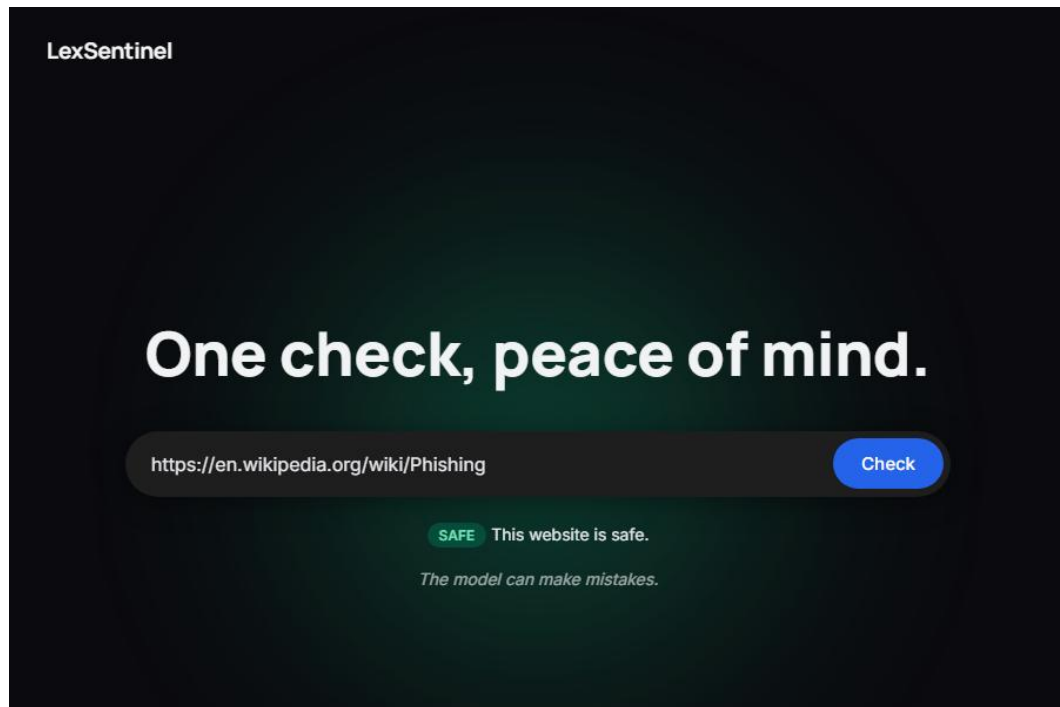


Figure 5.7 — SAFE verdict (allowlist stage)

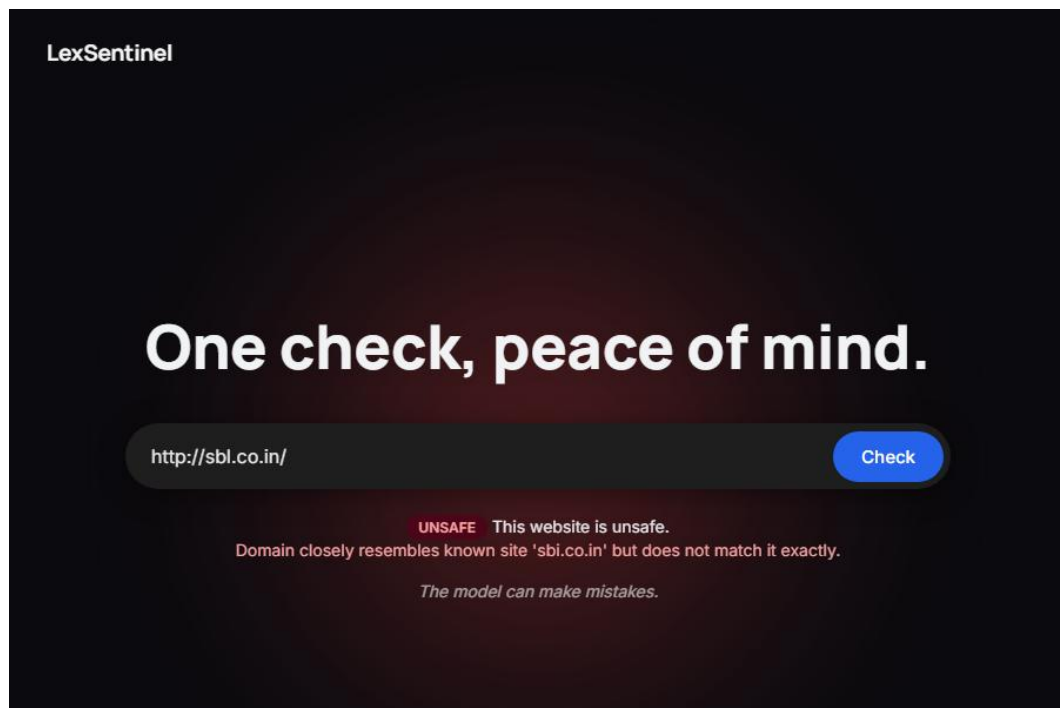


Figure 5.8 — UNSAFE verdict with typosquat note (`sbl.co.in` vs. `sbi.co.in`)

Figure 5.9 shows the INVALID input state (yellow glow, yellow badge) for text that does not resemble a URL at all, and Figure 5.10 shows the client-visible error state (blue glow) produced when the server rejects an over-length URL (>2048 characters) with a 422 response — the exact four-colour state machine (green / red / yellow / blue) documented as the target behaviour during this project's own end-to-end system test phase.

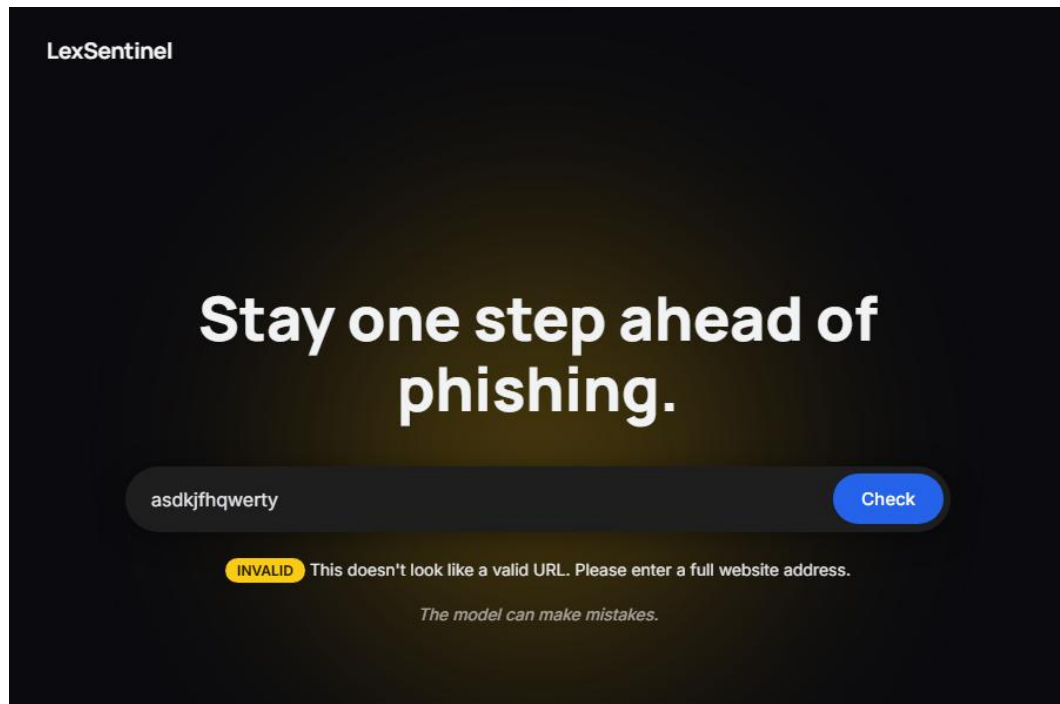


Figure 5.9 — INVALID input state

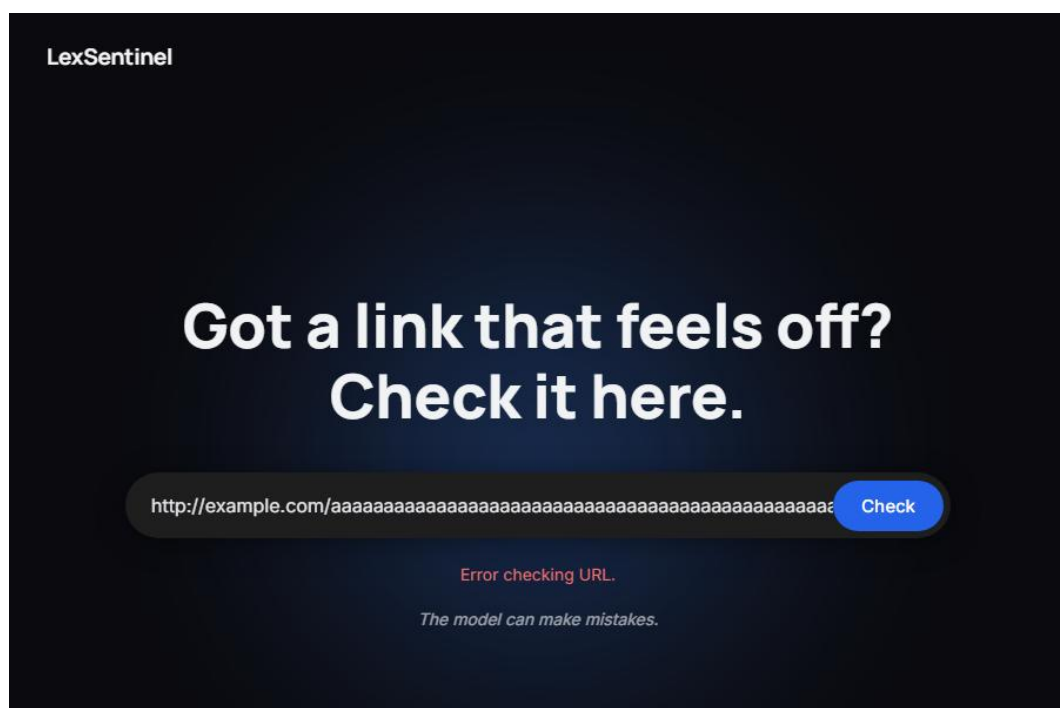


Figure 5.10 — Server-error state (422, over-length URL)

Figure 5.11 shows the developer-only bulk-checking page (/dev/bulk), which accepts a developer key, a file upload (.txt or .csv), or pasted URLs, and returns a downloadable results CSV.

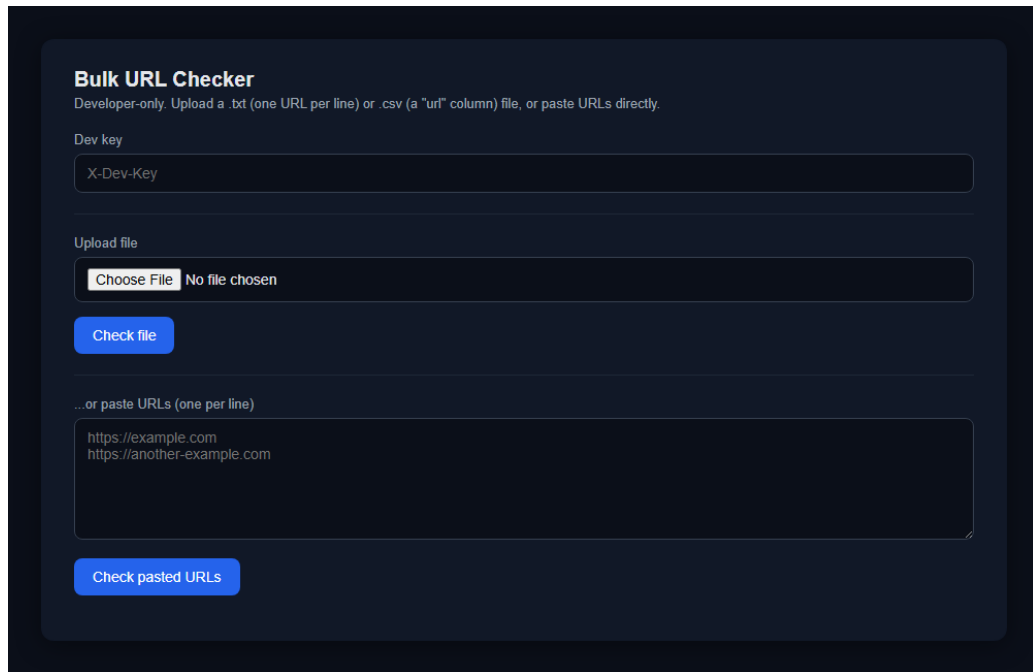
The image shows a web interface titled "Bulk URL Checker" with a subtitle "Developer-only. Upload a .txt (one URL per line) or .csv (a 'url' column) file, or paste URLs directly." It features three main input sections: 1) "Dev key" with a text input field containing "X-Dev-Key"; 2) "Upload file" with a "Choose File" button and the text "No file chosen", followed by a blue "Check file" button; 3) "...or paste URLs (one per line)" with a text area containing two example URLs, "https://example.com" and "https://another-example.com", followed by a blue "Check pasted URLs" button.

Figure 5.11 — Developer bulk-checking page, idle state

5.3 Data Preparation Strategy

Two distribution artefacts in the raw PhiUSIIL dataset (Section 3.3) required explicit correction, and the correction strategy in each case was structural — fixing the feature computation itself — rather than a one-off data patch, so that the fix applies to every future URL rather than only to a specific augmented example.

5.3.1 The `www.-prefix` artefact

Essentially 100% of PhiUSIIL's legitimate examples carry a `www.` prefix, while a majority of its phishing examples do not. A model trained on this distribution unmodified can use "starts with `www.`" as a strong proxy for "is legitimate" — a shortcut that fails immediately for real, well-known legitimate sites with no `www.` (`openai.com`, `stripe.com`, `discord.com`). The fix computes every feature on a `www.-stripped` copy of the host, implemented with a case-insensitive substring replacement on the raw URL string (not a lower-cased copy, since the path/query can legitimately carry meaningful case) so that "`HTTP://WWW.GOOGLE.COM/`" and "`http://google.com/`" produce identical count-based features.

5.3.2 The bare-homepage / trailing-slash artefact

PhiUSIIL's legitimate class is, as established in Section 3.3, essentially 100% bare domain-root URLs with no path — and a bare root with a trailing slash ("https://x.com/") and without one ("https://x.com") are the same URL to a browser but produce different raw slash-count features unless corrected. This project normalises the bare-root case only (a path's real slash count elsewhere is left untouched, since it does carry genuine signal) so the two forms are byte-identical at the feature level.

5.3.3 Augmentation to counteract the "any real path = phishing" shortcut

Structural fixes alone cannot teach a model that an ordinary content URL with a real path (a Wikipedia article, a government service page, a product listing) is typically legitimate, because the source dataset contains essentially zero such legitimate examples to learn from. This project's training pipeline (core/augmentation_data.py) adds real, search-sourced legitimate URLs with genuine paths, added only to the training split (the held-out PhiUSIIL test split is left untouched, so reported test metrics are never inflated by data the model was also shown at training time).

The augmentation set's own history is itself a methodological finding worth recording. Its first version (~69 URLs, replicated 40× in training) was shown, via a 100,000-URL evaluation conducted during an earlier audit pass, to cause outright memorisation rather than generalisation: a verbatim-augmented path on pandas.pydata.org scored 0.22% phishing probability, while a same-shape, unseen path on the different-but-structurally-similar numpy.org scored 99.94% — and two different paths on the very same domain (realpython.com), one augmented and one not, scored 0.40% and 99.86% respectively. The model had learned specific training strings, not the general shape of a legitimate content URL. The corrective principle adopted afterwards, and used for the remainder of this project, is diversity over replication: the augmentation set was expanded to 254 URLs across 215 distinct hostnames spanning more than 20 topic categories (government services, encyclopedic reference, cooking, outdoor recreation, personal finance, consumer tech reviews, sports news, online education, real estate, car maintenance, gardening, museums, university admissions, non-.com government/education TLDs such as .gov.uk and .nhs.uk, and more), with the replication multiplier correspondingly reduced from 40× to 8× — because high replication of a small set is precisely the mechanism that caused memorisation, not merely an efficiency knob.

A second, brand-agnostic augmentation set (core/wordplay_training_data.py) adds synthetically generated character-substitution and homoglyph phishing examples, paired with legitimate counter-examples built around brand names

that are themselves numeric or alphanumeric (so the model does not learn "any digit inside a domain name is suspicious" as an overly blunt shortcut).

5.4 Feature Optimization Methodology

Feature selection in this project was driven by three explicit constraints rather than an automated search procedure: every feature must be computable from the URL string alone (no network call), every feature must be defined identically at train and serve time (Section 5.3's structural-fix philosophy), and every feature's provenance must be auditable (no undocumented formula copied from a paper that cannot be reproduced from its description alone). Within those constraints, engineering iteration was itself the optimisation loop: the typosquat detector's core-extraction logic, corroboration gates, and generic-prefix exception list (Section 5.6 example) were each added or tightened in direct response to a confirmed false positive or false negative found through live testing against real domains, documented case by case in Chapter 7's implementation narrative. The one dimension-reduction step applied is on the text side: the character-n-gram TF-IDF vocabulary is capped at 2,000 features with a minimum document frequency of 3, which discards n-grams too rare to generalise while remaining small enough that the vectoriser's own vocabulary is a committed part of the versioned model artifact (Section 5.6), not an independently drifting external resource.

5.5 Machine Learning Model Selection

Three classifier families were trained and compared on the identical feature matrix (the 34 numeric features plus the shared TF-IDF text branch, over the identical train/test split and the identical augmentation), so that the comparison isolates the effect of model choice from every other variable: Logistic Regression (a linear baseline), Random Forest (a bagging ensemble), and XGBoost (a boosting ensemble). The full empirical results are presented in Section 7.7; the selection rationale is summarised here. Logistic Regression was included as the simplest possible baseline — if a linear model matched the ensembles, the added complexity of a tree ensemble would not be justified. Random Forest was included as a strong, well-understood bagging ensemble, and as the specific model family used in prior published work on this exact feature style (Sahingoz et al., 2019). XGBoost was selected as the final production model because it consistently produced the best combination of PhiUSIIL test-set metrics and, more importantly given this report's emphasis on honest evaluation (Chapter 1, Chapter 7), the best realistic held-out metrics — the measurement that most closely approximates real-world traffic.

5.6 Performance Evaluation Criteria

Every candidate model is evaluated on two deliberately different test sets, not one, reflecting this report's central methodological position (Section 1.2.3): a dataset's own held-out split measures in-distribution performance and can be systematically optimistic if that dataset carries collection artefacts (Section 3.3); a second, independently constructed "realistic held-out" set — fresh search-sourced legitimate URLs and freshly written phishing-shaped URLs, sharing no domain, path template, or generation logic with anything used in training or in the other test set — measures how the model behaves on traffic it was not shaped around.

Table 5.1 — Evaluation metrics used throughout Chapter 7

Metric	Definition	Why it matters here
Accuracy	Correct predictions / total predictions	Headline number, but misleading alone under class imbalance or distribution shift (Section 3.3)
Precision (phishing)	True phishing / all predicted phishing	Low precision means legitimate sites are wrongly blocked — direct user-trust cost
Recall (phishing)	True phishing / all actual phishing	Low recall means real phishing sites are missed — direct user-harm cost
ROC-AUC	Area under the ROC curve across all thresholds	Threshold-independent measure of ranking quality, used to compare models before a specific operating point is chosen
False Positive Rate (FPR)	Legitimate URLs wrongly flagged / all legitimate URLs	The single most important held-out metric for a usable security product; Chapter 7's central before/after comparison
False Negative Rate (FNR)	Phishing URLs wrongly cleared / all phishing URLs	The safety-critical metric; held at 0% throughout this project's held-out evaluations

All reported metrics are computed at a single, centrally defined decision threshold (`DECISION_THRESHOLD = 0.5`, `core/registry.py`) applied identically by the serving layer and the offline evaluation script — a constant that used to be defined separately (and briefly, inconsistently) in each, until a cleanup pass consolidated it to one location specifically so that a future threshold change cannot silently desynchronise reported metrics from actual production behaviour. This threshold is explicitly documented as an un-tuned default rather than a value chosen from a precision/recall curve at a target operating point — real, tracked future work rather than an implied guarantee (Chapter 8).

6. TECHNOLOGY USED

Every technology choice below was made against the non-functional requirements set out in Section 3.2 — in particular, low latency, offline capability, cross-platform portability, and auditability — rather than by default or familiarity alone.

6.1 Programming Language

Python 3.12 was used throughout the backend, machine-learning pipeline, and test suite. Python's dominance in the data-science and machine-learning ecosystem (scikit-learn, XGBoost, pandas all being Python-first libraries) made it the natural choice for a project that trains and serves a model from the same codebase; its readability was also weighted deliberately, since a security-relevant codebase (Chapter 7 documents recurring bug classes such as validation gaps and encoding mistakes) benefits from being easy for a reviewer to audit line by line.

6.2 Backend Framework — FastAPI

FastAPI was chosen over alternatives such as Flask or Django for three concrete reasons specific to this project. First, its native Pydantic integration gives request/response validation (URL length caps, bulk-request item limits) declared directly as type-annotated model fields rather than hand-written checks, which closes an entire class of the input-validation bugs this project's own audit history repeatedly found and fixed. Second, FastAPI automatically generates an OpenAPI schema and interactive documentation (/docs), which was used during development to exercise every endpoint by hand; the project also uses the ability to disable these in production (APP_ENV=production) specifically because they would otherwise disclose the shape of developer-only endpoints to an unauthenticated scanner. Third, a plain `def` (synchronous) FastAPI route runs automatically in a thread pool rather than blocking the single async event loop — a distinction that mattered concretely in this project, where an earlier `async def` bulk-file-upload handler was found (via security testing) to freeze the entire server, including the public single-check endpoint, for 35 seconds on one large upload; switching the handler to a plain `def` fixed this without any other code change.

6.3 Data Validation — Pydantic

Pydantic models (`CheckRequest`, `BulkCheckRequest`, `CheckResponse`, `BulkCheckResponse`) declare every request and response shape as typed fields with explicit length constraints, giving FastAPI enough information to reject malformed input (e.g. a URL longer than 2,048 characters, or a bulk request with more than 5,000 items) before any application code runs.

6.4 ASGI Server — Uvicorn

Uvicorn serves the FastAPI application over the ASGI protocol, and is the server actually run in production (Section 6.9's `Render` deployment invokes ``uvicorn app.main:app``). Its ``--reload`` flag was used throughout local development to shorten the edit-test cycle.

6.5 Machine Learning and Data Processing Stack

Table 6.1 — Machine learning / data processing libraries

Library	Role in this project
scikit-learn	Train/test splitting, the <code>ColumnTransformer</code> that fuses numeric features with the TF-IDF text branch, the <code>TfidfVectorizer</code> itself, and all reported classification metrics (precision, recall, F1, ROC-AUC, confusion matrix).
XGBoost	The production gradient-boosted-tree classifier (Section 4.2.3); also used for the model's own cross-platform-safe native (UBJSON) serialization format, separate from the sklearn preprocessing <code>joblib</code> file.
pandas	Loading the <code>PhiUSIIL</code> CSV, assembling the batched feature matrix (<code>extract_features_batch</code>), and driving the CSV export path for bulk results.
NumPy / SciPy	Underlying numeric array operations relied on transitively by scikit-learn and XGBoost.
joblib	Serializing the sklearn preprocessing half of the pipeline (<code>ColumnTransformer</code> + <code>TfidfVectorizer</code>) to disk, loaded back by <code>core/registry.py</code> at serve time.

6.6 Frontend Technologies

The public web UI (`app/static/index.html`) and developer bulk-checking page (`app/static/bulk.html`) are hand-written HTML, CSS, and vanilla JavaScript with no frontend build step, framework, or bundler — a deliberate choice for a project of this scope, where introducing a JavaScript framework (React, Vue) would add build tooling and a dependency surface disproportionate to two largely static pages whose only dynamic behaviour is a single `fetch()` call and DOM update. The UI uses the browser's native `fetch` API with an `AbortController` to guarantee a slow, superseded request can never overwrite the display of a faster, newer one — the direct fix for a confirmed "checked URL doesn't match what I typed" bug class.

6.7 Browser Extension — Manifest V3

The companion browser extension targets Manifest V3, the current Chrome/Chromium extension platform, using a background service worker (`background.js`) to intercept the `webNavigation` API and check each destination against the same backend the web UI uses, redirecting to an interstitial warning page when a site is flagged unsafe. Manifest V3 was used because it is the platform every current and future Chromium-based browser requires for new extensions (Manifest V2 is being phased out platform-wide); its documented limitation — that an extension can no longer synchronously pause navigation while an asynchronous check completes — is recorded candidly in this project's own documentation as a platform constraint every Manifest-V3 safe-browsing-style extension shares, not something specific to this build.

6.8 Testing Stack

`pytest` drives the automated test suite (12 test modules, Section 7.2), with `httpx` providing an in-process ASGI test client so API endpoints are exercised exactly as FastAPI would serve them, without needing a real network socket or a separately running server process.

6.9 Deployment and Version Control

- Git / GitHub — source control and the project's CI trigger surface.
- GitHub Actions — continuous integration, running the full `pytest` suite on every push and pull request.
- Render (`render.yaml` blueprint) — the production hosting target, chosen for its free tier and native support for a Python/uvicorn build+start command pair with no additional infrastructure configuration.

6.10 Dataset Source

The PhiUSIIL Phishing URL Dataset (Prasad and Chandra, 2023/2024) was obtained from Kaggle (mirrored on the UCI Machine Learning Repository), as documented in Chapter 3; it is the only external data dependency of the training pipeline.

6.11 Summary of Technology Choices

Table 6.2 — Full technology stack by layer

Layer	Technology
Language	Python 3.12
Web/API framework	FastAPI 0.139 + Pydantic 2.13
ASGI server	Uvicorn 0.50 (standard extras)
ML / data processing	scikit-learn 1.8, XGBoost 3.3, pandas 3.0, NumPy 2.4, SciPy 1.17, joblib 1.5
Frontend	Hand-written HTML5 / CSS3 / vanilla JavaScript (no build step)
Browser extension	Chrome Manifest V3 (service worker)
Testing	pytest 9.1, httpx 0.28
CI/CD	GitHub Actions
Hosting	Render (free tier)
Dataset	PhiUSIIL Phishing URL Dataset (Kaggle / UCI)

7.FRAMEWORK IMPLEMENTATION AND EXPERIMENTAL RESULTS

7.1 Development Environment

The system was developed and evaluated entirely on commodity, CPU-only hardware — no GPU is required at any stage, since gradient-boosted tree training and inference are CPU-bound workloads. Table 7.1 records the concrete environment the results in this chapter were produced on.

Table 7.1 — Development and evaluation environment

Component	Detail
Operating system	Windows 11 Pro
Python runtime	CPython 3.12.3 (pinned via runtime.txt for deployment parity)
Package manager	pip, with a project-local virtual environment (venv/)
Core ML libraries	scikit-learn 1.8.0, XGBoost 3.3.0, pandas 3.0.2, NumPy 2.4.4, SciPy 1.17.1, joblib 1.5.3
Web framework	FastAPI 0.139.0, Uvicorn 0.50.2, Pydantic 2.13.4
Testing	pytest 9.1.1, httpx 0.28.1
Version control	Git, with GitHub Actions CI running the full test suite on every push/PR
Compute profile	CPU-only; XGBoost trained with <code>n_jobs=-1</code> (all available cores), training completes in well under a minute at this dataset size

7.1.1 Iterative audit-and-upgrade development methodology

Rather than a single train-once-and-ship cycle, the system reported here was produced through a structured, phase-by-phase audit-and-upgrade pass, with every phase committed independently (giving a clean rollback point) and every phase ending with a full, green test-suite run before the next began. The automated test suite grew from 91 passing tests at the start of this pass to 131 at its conclusion (40 new tests), and every one of the 11 confirmed defects listed in Table 7.2 was verified live against a real, running instance of the application before being fixed — not merely inferred from reading the source.

Table 7.2 — Confirmed defects found and fixed during the audit pass (Phase 1)

#	Component	Defect found	Resolution
1	core/typosquat.py	Subdomain-prefix brand check flagged major real sites (outlook.live.com, id.atlassian.com, usa.philips.com, zoom.cisco.com) as phishing	Short generic cores excluded; a small set of legitimate common subdomain labels now requires corroborating evidence before flagging
2	app/static/bulk.html	Pasted URL lists used <code>split("\\n")</code> (a literal backslash-n), so multi-line paste never actually split	Fixed to a real newline-aware regex split
3	extension/popup.js	Crashed calling <code>.toUpperCase()</code> on a null verdict for unreachable pages (e.g. localhost)	Dedicated “can’t check” UI state for <code>status=”invalid”</code>
4	app/main.py /api/bulk-check	No input validation (unlike the single-check endpoint) — garbage lines received confident safe/unsafe verdicts	Same <code>is_valid_url</code> pre-filter applied to both paths
5	core/auth.py	Rate limiting keyed off the first (client-supplied, spoofable) X-Forwarded-For entry	Keyed off the last (platform-appended) entry instead
6-7	app/static/index.html	Enter key did nothing; non-2xx responses were silently swallowed by an accidental <code>TypeError</code>	Keydown handler added; explicit non-2xx branch surfaces the server’s own error detail
8	models/train.py	Metadata/pointer files written without explicit UTF-8 encoding	<code>encoding=”utf-8”</code> added to both writes
9	extension/options.js	Any string accepted as a backend URL, with no validation	<code>http(s)</code> URL format validated before saving
10	app/static/bulk.html	Results-table renderer assumed verdict is never null, crashing on invalid rows	Null-safe rendering for invalid/error rows
11	README.md / render.yaml	Deploy instructions referenced a non-existent <code>render.yaml</code> and a stale training-data path	<code>render.yaml</code> created; documentation corrected throughout

Later phases centralized the decision threshold to a single definition (Section 5.6), retrained and re-evaluated the model against both test sets (Section 7.5–7.8), added 29 further edge-case API tests (Section 7.2’s malformed-input coverage), performed a live end-to-end system test across all four UI states (Section 5.2.2), and closed with a full regression re-run and a stale-reference sweep of the codebase.

7.2 Dataset Loading and Inspection

The PhiUSIIL CSV (235,795 rows, ≈56 MB) was loaded with pandas and inspected before any modelling began. Two findings from this inspection directly shaped the rest of the pipeline and were verified against real, independently known URLs rather than assumed from the file’s column names:

- Label convention: the raw label column is 1 for legitimate and 0 for phishing — confirmed by checking a known German university domain (uni-mainz.de, mapped to label 1) and a known throwaway free-TLD domain (teramill.com/.gq, mapped to label 0). The project inverts this at load time (y

= 1 – raw_label) so that 1 = phishing consistently downstream, matching the intuitive convention used throughout the rest of the codebase and this report.

- Class balance: 134,850 legitimate rows (57.2%) versus 100,945 phishing rows (42.8%) — a mild imbalance, handled by XGBoost’s `scale_pos_weight` parameter rather than resampling.
- Structural artefacts: essentially 100% of legitimate rows are bare homepage URLs with a `www.` Prefix, addressed structurally as described in Section 5.3.

7.3 Data Preparation Results

Table 7.3 shows the exact row counts at each stage of the training-data pipeline for the currently deployed model (version 20260709T131117Z).

Table 7.3 — Training-data pipeline row counts (matches `models/artifacts/model_20260709T131117Z.metadata.json`)

Stage	Rows added	Running total
Raw PhiUSIIL dataset	235,795	235,795
Stratified 80/20 split → train portion	–47,159 (to test)	188,636 (train) / 47,159 (test, untouched thereafter)
+ Real benign-with-path augmentation (254 URLs × 8 replication)	+2,032	190,668 (train)
+ Wordplay/homoglyph phishing examples (996 URLs × 8 replication)	+7,968	198,636 (train)
+ Legitimate numeric-brand counter-examples (16 URLs × 40 replication)	+640	199,276 (train)

The final training set (199,276 rows) and held-out test set (47,159 rows) sum to exactly the 246,435 rows recorded in the shipped model’s metadata, and the test set itself is never touched by augmentation — every augmented row is added only to the training split, so PhiUSIIL test-set metrics in Section 7.6 are never inflated by data the model was also shown as “answers” during training.

7.4 Feature Selection Results

All 34 engineered numeric features (Table 3.5) plus the character 3–5-gram TF-IDF text branch (capped at 2,000 terms, minimum document frequency 3) were retained in the final feature set — no feature was found to be pure noise during development. To confirm which features the deployed model actually relies on, gain-based feature importances were extracted directly from the trained XGBoost booster. Table 7.4 lists the top 15 of 2,034 total features (34 numeric + 2,000 TF-IDF terms).

Table 7.4 — Top 15 features by XGBoost gain-based importance (extracted from the deployed model, version 20260709T131117Z)

Rank	Feature	Type	Importance (gain)
1	is_https	numeric	0.4606
2	num_special_chars	numeric	0.0833
3	num_digits	numeric	0.0638
4	url_length	numeric	0.0528
5	num_slashes	numeric	0.0134
6	"ing-" (char 4-gram)	TF-IDF	0.0109
7	"wp-" (char 3-gram)	TF-IDF	0.0096
8	"/blog" (char 5-gram)	TF-IDF	0.0095
9	num_dots	numeric	0.0093
10	"-be" (char 3-gram)	TF-IDF	0.0085
11	"/wp-" (char 4-gram)	TF-IDF	0.0084
12	"/how" (char 4-gram)	TF-IDF	0.0082
13	"-to" (char 3-gram)	TF-IDF	0.0081
14	tld_length	numeric	0.0062
15	"php" (char 3-gram)	TF-IDF	0.0059

Two observations follow directly from this table. First, `is_https` alone accounts for nearly half of total model gain — an interpretable and unsurprising result given that essentially all of PhiUSIIL's legitimate examples and the project's own augmentation set use HTTPS, while a majority of phishing examples do not; it is also flagged explicitly in Chapter 8 as a limitation, since an adversary shifting wholesale to HTTPS (increasingly cheap and automatic with free certificate authorities) would remove the model's single strongest signal. Second, several of the highest-ranked TF-IDF terms ("`/blog`", "`/how`", "`-to`", "`wp-`") are exactly the kind of ordinary content-URL substrings the Section 5.3 augmentation pass was designed to teach the model to treat as benign — direct evidence that the augmentation strategy is being used by the model as intended, not merely present in training data without effect.

7.5 Machine Learning Model Development and Evaluation

Three classifiers (Section 5.5) were trained on the identical feature matrix, identical train/test split, and identical augmentation, differing only in the learning algorithm itself: Logistic Regression (`max_iter=1000`, balanced class weights), Random Forest (200 trees, balanced class weights), and XGBoost (400 trees, max depth 6, learning rate 0.1, 90% row/column subsampling, `scale_pos_weight` matched to the training class ratio). Each was evaluated on

both the PhiUSIIL held-out test split and the fully independent realistic held-out set (19 fresh legitimate URLs across two categories never seen in training — freelance-tax guidance and puppy-obedience training — plus 15 freshly written phishing-shaped URLs spanning shipping/delivery, tax-refund, HR/payroll, subscription-renewal, cloud-storage-share, and IP-literal lure categories).

7.6 Performance Assessment

Model	PhiUSIIL test accuracy	PhiUSIIL test ROC-AUC	Held-out accuracy	Held-out precision	Held-out FPR	Held-out FNR
Logistic Regression	96.87%	0.9885	85.3%	75.0%	26.3% (5/19)	0.0%
Random Forest	98.05%	0.9887	79.4%	68.2%	36.8% (7/19)	0.0%
XGBoost (deployed)	98.34%	0.9926	94.1%	88.2%	10.5% (2/19)	0.0%

Table 7.5 — Head-to-head model comparison, identical features/splits/augmentation (all three trained and evaluated for this report)

The confusion matrix for the deployed XGBoost model on the PhiUSIIL test split (47,159 rows) is [[26,790 TN, 180 FP], [605 FN, 19,584 TP]] — a false-positive rate of 0.67% and false-negative rate of 3.00% on in-distribution data. All three models score well above 96% PhiUSIIL test accuracy and above 0.98 ROC-AUC; the models are genuinely distinguished only on the realistic held-out set (Table 7.5, right-hand columns), which is precisely the point of maintaining that second evaluation set (Section 5.6) — an evaluation restricted to PhiUSIIL's own test split alone would have shown three near-indistinguishable models and provided no basis for choosing among them.

7.6.1 Before/after: closing the held-out generalisation gap

Table 7.6 documents the single most consequential empirical result in this project: the effect of the Section 5.3.3 augmentation-diversity fix, measured on the identical XGBoost architecture and hyperparameters, with training data as the only variable changed.

Table 7.6 — Impact of augmentation-set diversification (Phase 3), same model architecture and hyperparameters throughout

Metric	Before (model 20260708T152549Z)	After (model 20260709T131117Z, deployed)
PhiUSIIL test accuracy	98.32%	98.33%
PhiUSIIL precision / recall	99.09% / 96.97%	99.09% / 97.00%
PhiUSIIL ROC-AUC	0.9925	0.9926
Realistic held-out accuracy	73.5%	94.1%
Realistic held-out precision	62.5%	88.2%
Realistic held-out recall	100%	100%
Realistic held-out FPR	47.4% (9 of 19)	10.5% (2 of 19)

PhiUSIIL test metrics are statistically unchanged (within noise) between the two models, while the realistic held-out false-positive rate falls 4.5-fold and recall remains perfect throughout. This is the clearest evidence in this report for its central claim (Chapter 1, Chapter 8): for this feature family and problem, training-data diversity — not model architecture, not hyperparameter tuning — was the binding constraint on real-world generalisation.

7.7 Model Comparison and Discussion

Logistic Regression's comparatively weak held-out performance (85.3% accuracy, 26.3% FPR) is consistent with the theoretical expectation set out in Section 4.1.2 and Section 4.2.4: a linear decision boundary cannot represent the conditional feature interactions (e.g., "many hyphens" is only suspicious alongside a credential-harvesting keyword and an unusual TLD, not alone) that separate legitimate content URLs from phishing URLs in this feature space, so it necessarily either over-generalises (flagging legitimate URLs that merely share a surface trait with phishing examples) or under-generalises. Random Forest performed worse than expected on the held-out set specifically (79.4% accuracy, 36.8% FPR) despite matching XGBoost closely on the PhiUSIIL test split — a pattern consistent with bagging's variance-reduction-only character (Section 4.1.2): it reduces overfitting to noise within the training distribution but does not, unlike boosting's sequential residual-fitting, receive any additional pressure to correct the specific systematic errors (Section 5.3's bare-homepage/www artefacts) that the augmentation set was designed to counteract. XGBoost's combination of shrinkage (learning rate 0.1), explicit L2/complexity regularisation (Section 4.2.3), and subsampling appears, on this evidence, to generalise the augmentation signal more effectively than either alternative, which is why it was selected as the deployed model (Section 5.5).

7.8 Best Model Identification and Result Interpretation

XGBoost (model version 20260709T131117Z) is identified as the best model on the combined evidence of Sections 7.6–7.7: the strongest PhiUSIIL test ROC-AUC (0.9926), the strongest realistic held-out accuracy (94.1%) and precision (88.2%), and — critically — a perfect 0% false-negative rate on both evaluation sets, meaning every phishing URL in either test set was correctly caught. Its two remaining held-out errors are both false positives, both individually explainable, and both documented rather than silently discarded:

- quickbooks.intuit.com/r/taxes/freelance-taxes/ (predicted phishing, probability 0.860) — a legitimate finance-software content URL in a topic category (freelance tax guidance) deliberately excluded from the training augmentation set specifically so it could serve as a genuine held-out test; the model has simply never seen a benign example combining a finance-adjacent domain with this path shape.
- doggoneproblems.com/miles/ (predicted phishing, probability 0.990) — an unaugmented, low-recognition domain with a short, generic path segment and no training-time analogue.

Both are consistent with the single limitation this report identifies repeatedly: residual false positives concentrate on legitimate URLs whose domain/path combination was not represented anywhere in training, not on any structural flaw in the model architecture — reinforcing that further improvement is most likely to come from continuing to expand the augmentation set's diversity (Chapter 8), the exact lever that already produced the 4.5× held-out FPR improvement documented in Section 7.6.1, rather than from a different classifier or additional feature engineering.

7.8.1 Automated regression test suite

Independent of model metrics, the system's correctness is guarded by an automated test suite of 1,645 lines across 12 modules, all passing at the time of writing (131 tests total, up from a baseline of 91 before the audit pass described in Section 7.1.1).

Table 7.7 — Automated test suite composition

Test module	Focus
<code>test_regression_known_sites.py</code>	CI release gate — known-legitimate sites (perplexity.ai, discord.com, india.gov.in, icici.bank.in, and more) plus synthetic suspicious-structure cases; must pass before any deploy
<code>test_feature_parity.py</code>	Guards against training and serving feature logic ever forking apart
<code>test_api_edge_cases.py</code>	29 tests: boundary lengths, non-Latin/percent-encoded/injection-shaped input, malformed JSON, gov/edu compound-TLD domains
<code>test_bulk_check.py</code>	Bulk JSON/file endpoints, CSV export correctness
<code>test_malformed_urls.py</code>	Structurally broken URL strings

Test module	Focus
test_security_fixes.py	Rate-limit spoofing, CSV-injection, auth gating
test_wordplay.py	Homoglyph/leetspeak detection primitives
test_extension_popup_bugs.py	Browser-extension popup null/edge states
test_frontend_regressions.py	Web-UI race-condition and rendering fixes
test_p1_fixes.py ... test_p4_fixes.py	Regression tests encoding an earlier hardening pass, predating this project's current FastAPI architecture

8. CONCLUSION AND FUTURE WORK

8.1 Project Conclusions

This project set out to design, implement, and — most importantly — honestly evaluate a phishing URL detection system built entirely on URL-string analysis, with no page fetch and no third-party reputation dependency in its decision path. That goal was met: LexSentinel combines a fast, explainable deterministic stage (blocklist, allowlist, Damerau–Levenshtein-based typosquat detection, and Unicode-confusable-aware wordplay detection) with an XGBoost classifier trained on 34 engineered lexical/structural features plus a character-n-gram TF-IDF representation of the URL path and query, and is exposed through a working web interface, a scriptable developer bulk-checking tool, and a Manifest V3 browser extension that protects a user automatically during ordinary browsing.

The project's more important conclusion, however, is methodological. Reporting a single accuracy number from a model's own source-dataset test split — the practice this report found repeated throughout the phishing-detection literature (Chapter 2) — can be dangerously misleading when that dataset carries collection artefacts a model can exploit without learning the underlying phenomenon. This project's own initial model scored 98.3% on PhiUSIIL's test split while missing nearly half of the legitimate URLs in an independently constructed, realistic held-out set (47.4% false-positive rate). Recognising and closing that specific gap — through targeted, evidence-driven training-data augmentation rather than a change of algorithm — is the single most consequential piece of engineering work in this project, and the held-out false-positive rate it produced (10.5%, a $4.5\times$ improvement, with recall held at a perfect 100% throughout) is this report's central empirical result.

8.2 Key Outcomes

- A single, canonical feature-extraction function (`core/features.py`) shared identically between training and serving, eliminating an entire historical class of train/serve-skew bugs.
- A defense-in-depth decision pipeline in which the highest-confidence cases (known brands, known-bad domains, brand typosquats) are resolved by explainable, deterministic rules rather than a black-box probability, with the ML model reserved as a general-purpose fallback.
- A genuinely held-out, disjoint-from-training evaluation methodology that exposed a 47.4% real-world false-positive rate invisible to the source dataset's own test split, and a documented, reproducible fix (training-data diversification) that cut it to 10.5% without touching the model architecture.

- A rigorous, evidence-based empirical comparison of three classifier families (Logistic Regression, Random Forest, XGBoost) on identical features and data, confirming with this project's own data — not only literature precedent — that boosted trees generalise best to unseen traffic for this feature family.
- A working, deployable product across three surfaces (web UI, developer bulk API, browser extension), verified end-to-end, not only as an offline notebook model.
- A 131-test automated regression suite in which every previously confirmed defect or false positive/negative is encoded as a permanent, executable test case.

8.3 Current Constraints

Several limitations are deliberate design trade-offs, and several are genuine, acknowledged gaps; both kinds are recorded here rather than left implicit.

- No live network, WHOIS, DNS, TLS-certificate, or page-content signal is used, by design (Section 1.2.2, Section 3.4.1) — this is a deliberate latency/WAF-safety trade-off, but it also means the system cannot use signals (e.g. domain age) that a content-fetching competitor could.
- The decision threshold (0.5) is a centrally defined but un-tuned default (Section 5.6), not a value chosen from a precision/recall curve at a deliberate operating point, and the system currently produces only a binary safe/unsafe verdict with no "uncertain" middle band.
- The realistic held-out evaluation set is small ($n = 34$); each misclassification shifts its reported rate by roughly three percentage points, so its absolute numbers should be read as directional evidence of generalisation, not a tight confidence interval.
- Gain-based feature analysis (Section 7.4) shows the deployed model relies heavily on a single feature (`is_https`, $\approx 46\%$ of total gain); an adversary shifting wholesale to free, automatic HTTPS certificates would remove the model's single strongest signal, though the deterministic typosquat/wordplay stage and the model's remaining features would still apply.
- The allowlist, blocklist, and protected-brand reference set are small, manually seeded lists (Section 3.2), explicitly documented as placeholders for a live feed (Tranco Top 1M, PhishTank/OpenPhish) in production, and the typosquat detector's current $O(\text{list-size})$ per-check comparison would need a smaller curated reference set (rather than the full allowlist) if that list grows to Tranco scale.
- One held-out legitimate URL (`nerdwallet.com`) currently overlaps between the training augmentation set and the held-out evaluation set — a pre-existing, documented data-hygiene issue that should be resolved the next time the held-out set is regenerated, to keep the two sets fully disjoint.

8.4 Future Scopes

- Threshold tuning and a three-way verdict: derive an operating threshold from a precision/recall curve against a target false-positive budget, and introduce a "suspicious" middle verdict between safe and unsafe for scores in an ambiguous band, rather than a hard binary cutoff.
- A substantially larger, continuously regenerated realistic held-out set, ideally sampled on a rolling basis from live, recent phishing feeds and fresh legitimate web traffic, to tighten confidence in the generalisation metrics beyond the current $n = 34$.
- Continued augmentation-set diversification — the single lever that produced this project's largest measured improvement (Section 7.6.1) — targeting the specific categories this report's own held-out misses identify (finance-adjacent content URLs, low-recognition domains with generic path segments).
- Live feed integration for the allowlist/blocklist/protected-brand configuration (Tranco Top 1M, PhishTank, OpenPhish, URLhaus), replacing the current seed lists with a scheduled refresh job, as already documented as the intended production path in the project's own configuration files.
- A feature-reliance audit and mitigation for the `is_https` dominance finding (Section 7.4) — for example, explicitly down-weighting or interacting `is_https` with other features during training so the model does not collapse toward a near-single-feature decision rule as HTTPS adoption among phishing sites rises.
- Formal explainability output per verdict (e.g. SHAP values for the model stage, surfaced alongside the existing stage/note fields) so a flagged user is shown not just a confidence score but the specific features that drove it — extending the transparency principle this project already applies to its rule-based stages.
- Extending the browser extension beyond Manifest V3 Chromium browsers to Firefox's WebExtensions equivalent, broadening real-world protection coverage.

9. REFERENCES

- [1] Prasad, A., & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers & Security*, 136, 103545.
- [2] Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357.
- [3] Le, H., Pham, Q., Sahoo, D., & Hoi, S. C. H. (2018). URLNet: Learning a URL representation with deep learning for malicious URL detection. *arXiv preprint arXiv:1802.03162*.
- [4] Zhang, Y., Hong, J. I., & Cranor, L. F. (2007). CANTINA: A content-based approach to detecting phishing web sites. In *Proceedings of the 16th International Conference on World Wide Web (WWW '07)* (pp. 639–648). ACM.
- [5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)* (pp. 785–794). ACM.
- [6] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [7] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- [8] Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8), 707–710.
- [9] Damerau, F. J. (1964). A technique for computer detection and correction of spelling errors. *Communications of the ACM*, 7(3), 171–176.
- [10] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- [11] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

- [12] Shannon, C. E. (1948). A mathematical theory of communication. The Bell System Technical Journal, 27(3), 379–423.
- [13] Unicode Consortium. Unicode Standard Annex #15: Unicode Normalization Forms.
- [14] Anti-Phishing Working Group (APWG). Phishing Activity Trends Report (quarterly series).
- [15] Prasad, A., & Chandra, S. PhiUSIIL Phishing URL Dataset. Kaggle / UCI Machine Learning Repository.
- [16] Google LLC. Google Safe Browsing — service overview and developer documentation.
- [17] Microsoft Corporation. Microsoft Defender SmartScreen — overview documentation.
- [18] PhishTank (OpenDNS / Cisco). Community phishing-verification database and API.
- [19] OpenPhish. OpenPhish phishing intelligence feed — service documentation.
- [20] Ramírez, S. FastAPI — framework documentation.
- [21] The pandas development team. pandas: powerful Python data analysis toolkit — documentation.
- [22] OWASP Foundation. OWASP Top Ten — web application security risks.
- [23] Google Chrome Developers. Chrome Extensions — Manifest V3 documentation.